

ANALYSIS AND PREDICTION OF WEATHER PATTERNS USING SATELLITE OBSERVATIONS AND MACHINE LEARNING MODELS

NAME OF THE STUDENTS

AGNIK GHOSH
ABHRANIL CHAKRABORTY
SANTANU MONDAL
CRASIVA CHAUDHURY

Under the guidance of
Dr. Tuhina Halder



Department of Electronics and Communication Engineering

St. Thomas' College of Engineering and Technology

4 D.H. Road, Kolkata 700023

July, 2026

Vision of the Department

- To build a strong teaching and research environment to cater to the manpower needs in Industrial and Academic domains of the rapidly growing Electronics and Communication Engineering.

Mission of the Department

- To produce certified industry-ready professional in Electronics and Communication Engineering, through innovative educational programs incorporating laboratory practices and project-based teaching-learning processes, in a modern environment.
- To create knowledge base of advanced technologies through research in the area of Electronics and Communication, for competitive and sustainable development of the country.
- To groom the department as a learning center to inculcate advancement of technology in Electronics and Communication Engineering with social values and environmental awareness.

Program Specific Outcome (PSOs)

After completion of program graduate engineer would have:

- **PSO1. Professional skills:** Solve real life problems relevant to Electronics and Communication Engineering in various areas, like VLSI, Nano-technology, Advanced Communications, Signal Processing, Embedded Systems, IoT and applications involving AI & ML.
- **PSO2. Competency:** Demonstrate competence in qualifying exams and pursue successful careers in employment, research and higher education at state, national, and international levels.

Program Outcome

Engineering Graduates will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
 2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
 3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
 4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
 5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
-

6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and Teamwork:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team to manage projects and in multidisciplinary environments.
12. **PO12: Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Course Outcomes for Final Year Project

After completion of the project, student will be able to:

CO no.	Outcome Statements	Bloom's Level
CO1	Apply technical knowledge in the solution of complex real-life problems related to public health and safety, culture, society, and environment	3
CO2	Review research literature and critically evaluate research-based knowledge to identify and formulate engineering problems effectively	5
CO3	Design innovative solutions for complex engineering problems that can be published as research papers or developed into marketable products in the future	6
CO4	Apply modern tools to design and develop solutions to problems	3
CO5	Evaluate societal, health, safety, legal and cultural issues relevant to project and the responsibilities relevant to the professional engineering practice.	5
CO6	Evaluate the impact on the environment of working project	5
CO7	Apply ethical principles and commit to ethics and responsibilities related to engineering practice.	3
CO8	Apply effective interpersonal and leadership skills to function as a member or leader of a team in engineering practice	3
CO9	Apply effective communication skills to engage professionally with team members, superiors, and society at large	3
CO10	Apply effective planning, management, and financial control skills to execute projects successfully as a member or leader of a team.	3
CO11	Apply the knowledge acquired during the project, in future higher studies or professional job.	4
CO12	Analyze real-life problems related to Electronics and Communication Engineering to find the solutions in areas like advanced communications, VLSI, Embedded Systems, and AI/ML	4

CO13	Evaluate emerging technologies and demonstrate professional competence for success in careers, research, and higher education.	5
CO14	Independently analyze and adapt to evolving software frameworks, library updates, and fluctuating climate datasets for continuous model enhancement in future endeavors.	4

Bloom's Level: Remember = 1, Understand = 2, Apply = 3, Analyze = 4, Evaluate = 5, Create = 6

CO-PO Matrix

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO
CO1	3	2	2			3	2						2	
CO2		3		3										2
CO3	2		3	2									3	3
CO4					3								3	
CO5						3								
CO6							3							
CO7								3						
CO8									3					
CO9										3				
CO10											3			
CO11												3		3
CO12	3	2			2								3	
CO13												3		3
CO14												2		2

Program level Course-PO matrix

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO	2	2	2	2	2	3	2	3	3	3	3	2	3	3

Project Planning

The project followed a structured four-phase development lifecycle to ensure timely delivery and high predictive accuracy:

- **Phase 1: Data Acquisition & Preparation (Weeks 1–3):** Collection of multi-year (2022–2025) meteorological datasets, followed by rigorous data cleaning and chronological sorting to prevent future-data leakage.
- **Phase 2: Feature Engineering & Model Architecture (Weeks 4–7):** Development of the 79-feature pipeline (lags, rolling averages, cyclical time encoding) and the design of the two-stage GBM Hurdle Model.
- **Phase 3: Training, Tuning & Validation (Weeks 8–10):** Hyperparameter optimization for the classifier/regressor and threshold calibration using the validation dataset.
- **Phase 4: Integration & Documentation (Weeks 11–12):** Deployment of the modular inference pipeline, generation of the visual dashboard, and final project report compilation.

Financial Analysis

Category	Description	Estimated Cost
Data Acquisition	Utilization of Open-Meteo & OpenWeatherMap APIs (Free tiers)	₹0
Software & Tools	Python, Scikit-learn, Pandas, Matplotlib (Open Source)	₹0
Compute Resources	Local workstation usage (CPU/RAM optimization)	₹0 (In-house)
Hardware	Sensors, Microcontroller, Wires, and Power	~₹440 – ₹660
Miscellaneous	Printing, binding, and incidental research costs	Minimal
Total		~₹440 – ₹660

Certificate

Department of Electronics and Communication Engineering

St. Thomas' College of Engineering and Technology

This is to certify that the project entitled “**Analysis and prediction of weather patterns using satellite observations and machine learning models**”, has been carried out by

Agnik Ghosh (12200322044),
Abhranil Chakraborty (12200322025.),
Santanu Mondal (12200322013),
Crasiva Chaudhury (12200322050),

under my guidance during the year **July 2025 to June 2026** and accepted for partial fulfillment of the requirement of the Degree of Bachelor of Technology of Maulana Abul Kalam Azad University of Technology, West Bengal, India

.....
Signature of Project guide

Dated:

Acknowledgement

We would like to express our profound gratitude to **Prof Dr. Prasun Choudhury (HOD)** of Electronics and Communication Engineering, and to **Prof Dr. Shila Ghosh, (Principal)**, for providing us this wonderful opportunity to work on this project titled "Analysis and Prediction of Weather Patterns Using Satellite Observations and Machine Learning Models."

We would also like to express our special thanks to our mentor, **Prof Dr. Tuhina Halder**, for her vital cooperation and help in ensuring the successful completion of our project. Your useful advice and suggestions were helpful to us during the project completion, and in this respect, we are eternally grateful to you.

Finally, we would convey our sincere thanks to all the teachers, project lab assistants, our friends, and supporters. Without them, the task would not have been accomplished in such a timely manner.

Thanking You,

Agnik Ghosh
(12200322044)

Abhranil Chakraborty
(12200322025)

Santanu Mondal
(12200322013)

Crasiva Chaudhury
(12200322050)

Contents

Chapter No.	Chapter Name	Page No.
1.	Introduction Project Overview Objectives & Applications	1-3
2.	Background of the Project Literature Survey	4-6
3.	Details of the Project Component List Description Flow Chart Algorithms Programs	6-22
4.	Results Graphs Result Analysis	23-40
5.	Conclusion (including Discussion & Future Works)	41-42
6.	Reference	42-43
7.	Appendices	44-47

1. INTRODUCTION

Accurate rainfall prediction is very important, especially in cities like Kolkata, where heavy rainfall during the monsoon can cause waterlogging, traffic jams, and damage to infrastructure. Predicting rainfall is difficult because weather conditions change quickly. This project aims to solve this problem using Machine Learning to predict whether it will rain and, if it does, how much rainfall (in mm/hr) is expected.

The project uses a two-stage Machine Learning model based on Gradient Boosting. In the first stage, a Gradient Boosting Classifier predicts whether rain will occur or not. If rain is predicted, the second stage uses a Gradient Boosting Regressor to estimate the amount of rainfall.

To improve prediction accuracy, the model uses weather data collected over several years. Important weather factors such as temperature, humidity, and air pressure are used to create useful features like moving averages, previous weather conditions, dry spell duration, and time-based patterns. These features help the model understand weather trends more effectively.

The model is trained using weather data from 2022 to 2024 and tested using 2025 data that it has never seen before. This helps check whether the model works well in real-life situations.

Overall, this project provides an accurate and practical rainfall prediction system that can support early weather warnings, agriculture, flood management, and urban planning.

1.1 PROJECT OVERVIEW:

Project Objectives: The primary objective of this project is to develop a Machine Learning-based rainfall prediction system that can determine whether rainfall is likely to occur in Kolkata. If rainfall is predicted, the system further estimates the expected rainfall intensity in millimetres per hour (mm/hr) using historical weather observations.

Two-Stage Prediction Model: The proposed system follows a two-stage prediction approach based on Gradient Boosting algorithms. In the first stage, a classification model predicts whether rainfall is expected (Rain or No Rain). If rainfall is detected, a regression model is activated in the second stage to estimate the expected rainfall amount accurately.

Dataset Used: The Machine Learning models are trained using historical weather data collected from 2022 to 2024. To evaluate the model's ability to generalize to new data, its performance is tested using weather data from 2025, which is completely excluded from the training process.

Feature Engineering: Historical weather parameters, including temperature, humidity, and air pressure, are processed to generate 79 informative features. These features include lagged weather observations, moving averages, and dry-period duration, enabling the model to capture temporal weather patterns and improve prediction accuracy.

Practical Applications: Accurate rainfall prediction can support flood management, agricultural planning, urban infrastructure management, and early weather warning systems. The developed system can assist government agencies and local authorities in making timely decisions and improving preparedness during heavy rainfall events in Kolkata.

1.2 OBJECTIVES:

To collect and preprocess historical weather data of Kolkata (2022–2025) by using important weather parameters such as **temperature, humidity, and air pressure**.

To develop a **two-stage Machine Learning model** where the first model predicts **whether it will rain or not**, and the second model estimates the **amount of rainfall (mm/hr)** if rain is expected.

To improve prediction accuracy by creating useful features from the weather data, such as **previous weather conditions, moving averages, dry spell duration, and time-based patterns**.

To train the model using weather data from **2022 to 2024** and test it with **2025 data** to evaluate its performance on unseen data using error metrics like **MAE (Mean Absolute Error)** and **RMSE (Root Mean Square Error)**.

To provide accurate rainfall forecasts that can help **flood management, agriculture, urban planning, and early weather warning systems**.

To build a flexible and scalable prediction system that can be improved in the future by adding more weather parameters or using advanced Machine Learning techniques.

1.3 APPLICATIONS

1. Urban Flood Management

The rainfall prediction system helps the Kolkata Municipal Corporation (KMC) prepare for heavy rainfall in advance. By estimating the expected rainfall intensity, authorities can operate drainage pumps early and take preventive measures in flood-prone areas such as Behala and Mukundapur, reducing the chances of severe waterlogging.

2. Early Warning and Disaster Management

The model can predict the possibility of rainfall before it occurs, allowing disaster management teams to issue timely weather alerts. Early warnings help emergency services, local authorities, and electricity departments prepare for heavy rain and reduce risks such as flooding and electrical accidents.

3. Traffic and Transportation Management

Heavy rainfall often causes traffic congestion and waterlogging in Kolkata. Accurate hourly rainfall forecasts can help traffic authorities plan diversions, manage vehicle movement, and inform commuters about affected routes, making travel safer and more efficient.

4. Agriculture and Water Resource Management

Farmers can use rainfall predictions to plan irrigation, protect crops from excessive rainfall, and improve agricultural productivity. The forecasts can also help manage water levels in the East Kolkata Wetlands, supporting better water conservation and environmental protection.

5. Climate Research and Future Urban Planning

The project provides a flexible Machine Learning framework that can be improved by adding new weather data in the future. It can support climate research, help scientists study changing weather patterns, and assist city planners in developing better infrastructure to handle future climate challenges.

2. BACKGROUND OF THE PROJECT

Kolkata experiences heavy rainfall during the monsoon season, and the weather can change very quickly. Sudden rainfall often causes waterlogging, traffic congestion, damage to infrastructure, and other problems that affect daily life. Although weather forecasting has improved over the years, accurately predicting when it will rain and how much rain will fall in a specific area is still a difficult task.

To address this problem, this project uses Machine Learning to improve rainfall prediction. Historical weather data from 2022 to 2024, including temperature, humidity, and air pressure, is used to train the model. A two-stage Gradient Boosting approach is applied, where the first model predicts whether rain is likely to occur, and the second model estimates the expected rainfall amount in mm/hr.

The main aim of this project is to provide more accurate and reliable rainfall forecasts that can support flood management, disaster preparedness, agriculture, and urban planning, helping authorities and the public make better decisions before heavy rainfall occurs.

2.1 LITERATURE SURVEY

1. **D. Sharma and P. Rattan [1]** compared Random Forest and XGBoost for regional rainfall forecasting using historical meteorological data. They found that while Random Forest effectively mitigates overfitting through bagging, XGBoost achieves superior computational efficiency and smaller prediction errors via sequential optimization. However, both models are vulnerable to data quality anomalies and sudden climate shifts.
2. **J. Puttanapura and C. K. K. Reddy [2]** implemented an improvised Random Forest Classifier to predict localized rainfall over Taiwan using temperature, humidity, and pressure features. The ensemble approach successfully handled high-frequency atmospheric noise and feature non-linearity. Nevertheless, managing a large number of deep decision trees creates structural overhead, and accuracy slightly decreases during extreme, low-frequency downpours.
3. **A. Jain [3]** explored Gradient Boosting algorithms to enhance the accuracy of short-term weather forecasts. The study demonstrated that sequential modeling of weak decision trees maps complex interactions with high coefficients of determination (R^2). Conversely, the framework requires intensive hyperparameter tuning and is highly susceptible to overfitting if learning duration is unregulated.
4. **P. Sahu, S. Kumar, and R. K. Yadav [4]** introduced an Explainable AI-Powered (XAI) machine learning approach for heavy rainfall prediction. This model supplements predictions with interpretability layers to uncover how variables like moisture convergence influence alerts, thereby boosting trust for early warning networks. However, adding XAI elements increases structural complexity and real-time processing demands.
5. **R. P. Utture, A. K. Biju, Y. V. S. Murthy, L. R. Cenkeramaddi, and N. Kumari [5]** built a weather forecasting framework that utilizes machine and deep learning to process satellite cloud images. The convolutional layers effectively capture complex spatial patterns and cloud densities without manual feature engineering. Despite these advantages, the model requires large training datasets and faces performance drops under poor visibility or sensor noise.
6. **W. Zou, J. Ji, Y. Wang, J. Wang, Y. Qian, and J. Liu [6]** proposed a hybrid Convolutional LSTM network embedded with a Self-Attention Mechanism to forecast

extreme weather. The architecture extracts spatiotemporal features over long sequence lags, using the self-attention block to prioritize critical anomalies. The primary trade-offs include prolonged training cycles, high complexity, and a risk of structural overfitting.

7. **Y. Liu and S. Cai [7]** reviewed advancements in weather forecasting driven by integrating AI with satellite imagery. They noted that computer vision and deep learning data fusion enable high-resolution global coverage and faster early warnings. Nonetheless, domain-wide challenges persist regarding data transmission bottlenecks and lack of standardization across separate sensor platforms.
8. **A. Saxena, N. Verma, and K. C. Tripathi [8]** surveyed traditional Artificial Neural Networks (ANN) and backpropagation algorithms applied to weather forecasting. They demonstrated that hidden neuro-computing layers efficiently approximate non-linear variables like minimum temperature. However, conventional ANNs suffer from slow convergence rates, local minima traps, and difficulty tracking long-term sequential dependencies.
9. **M. M. Hassan *et al.* [9]** developed a robust rainfall prediction system using an Australian dataset to support flood and drought preparedness. After rigorous preprocessing and IQR outlier removal, they tested multiple algorithms, revealing that an Artificial Neural Network (ANN) achieved the highest predictive accuracy (91%). The primary limitation is a heavy operational reliance on the completeness of historical data streams.
10. **A. E. Abdishakur, A. M. Tahlil, and A. A. Aden [10]** proposed a hybrid LSTM-GBM deep learning architecture for precipitation nowcasting. The LSTM gate structures extract temporal sequences from temperature and humidity arrays, which are then optimized by a Gradient Boosting Machine (GBM) layer to minimize prediction bias. This combined design yields near-perfect classification performance but increases algorithmic complexity and hyperparameter dependencies.
11. **B. K. Cahyono *et al.* [11]** designed an automated multivariate rainfall prediction framework over the Gajahwong watershed in Indonesia. Using the Google Earth Engine to capture CHIRPS data and eleven environmental covariates filtered through a digital Butterworth function, they optimized models via a Tree-Structured Parzen Estimator. Support Vector Regression (SVR) achieved the highest accuracy ($R^2 = 0.948$), though the model output is restricted to the temporal domain rather than spatial mappings.
12. **A. Rahman *et al.* [12]** developed a real-time rainfall prediction framework for smart city grids using a fuzzy logic data fusion layer. The system combines independent probability streams from Decision Trees, Naïve Bayes, KNN, and SVM models trained on a 12-year dataset. The fused stream provides robust early warning red-alerts, though its performance is vulnerable to physical sensor node malfunctions.
13. **L. Bhaskar, S. M. N, V. S, and T. Anand [13]** applied data-driven machine learning models in MATLAB to predict annual precipitation in the Andaman and Nicobar Islands. Comparing Tree-Based Regression with Linear Regression, they showed that tree architectures handle noisy, non-linear atmospheric changes with higher precision. However, performance degrades when facing extreme regional climate variability.
14. **M. M. Hassan *et al.* [14]** implemented an optimized machine learning framework using Python and SPSS to analyze regional precipitation profiles. Evaluating Naïve Bayes, SVM, and Random Forest models, they found that morning humidity and cloud height are the strongest predictors. While feature selection enhanced station accuracy, the model exhibits decreased stability when encountering unprecedented climate anomalies.
15. **A. E. Abdishakur, A. M. Tahlil, and A. A. Aden [15]** evaluated an integrated spatiotemporal model coupling LSTM blocks with a stage-wise Gradient Boosting Machine (GBM) for short-term nowcasting. The sequential vectors extracted by the LSTM layer feed directly into the GBM regressor to iteratively correct residual prediction errors.

The hybrid structure optimized the area-under-the-curve (AUC) but demands prolonged computational training cycles.

16. **B. K. Cahyono *et al.* [16]** investigated a remote sensing pipeline leveraging the Google Earth Engine to model tropical daily rainfall. After applying Min-Max scaling and a python-based Butterworth filter to satellite atmospheric arrays, they validated SVR, RF, GBR, and DNN setups. SVR delivered the highest cross-validation stability, though it lacks multi-dimensional spatial domain forecasting outputs.
17. **A. Rahman *et al.* [17]** engineered a multi-stage real-time weather forecasting framework powered by supervised machine learning fusion for automated smart cities. By aggregating the independent outputs of four baseline classifiers (DT, NB, KNN, SVM) through fuzzy membership rules, the system creates an accurate early-warning network. However, data ingestion relies strictly on sensor grid integrity, leaving it vulnerable to hardware faults.
18. **L. Bhaskar, S. M. N, V. S, and T. Anand [18]** explored tree-based machine learning regression structures in MATLAB using historical variables like humidity and barometric pressure. Quantitative scatter plots proved that tree configurations accurately map nonlinear climate transitions where classic linear equations fail, though severe dataset noise increases overall optimization complexity.

3. DETAILS OF THE PROJECT

3.1 Details

Project Title: Analysis and Prediction of Weather Patterns Using Satellite Observations and Machine Learning Model.

Domain: Machine Learning, Weather Forecasting, and Time Series Analysis.

Target Location: Kolkata, India, using historical weather data collected over three years (2022–2024).

Objective: The main objective of this project is to predict whether rainfall will occur and, if rain is expected, estimate the amount of rainfall (in mm/hr) using historical weather data.

Methodology: The project follows a two-stage Machine Learning approach.

Stage 1 (Classification): The first model predicts whether rainfall is likely to occur (Rain or No Rain).

Stage 2 (Regression): If rainfall is predicted, the second model estimates the expected rainfall amount (mm/hr).

Key Features:

- Uses historical weather information such as temperature, humidity, and air pressure.
- Generates meaningful features from previous weather conditions and time-based patterns to improve prediction accuracy.

- Uses only historical weather data for model training to prevent data leakage and ensure reliable predictions.
- Evaluates the model using unseen test data to measure its real-world prediction performance.

3.2. Component List

Software and Libraries Used

Programming Language: Python 3.x

Data Processing: **Pandas** and **NumPy** are used for handling, cleaning, and analyzing weather data.

Machine Learning: **Scikit-learn** is used to build the Gradient Boosting classification and regression models and to evaluate their performance.

Model Storage: **Joblib** is used to save and load the trained machine learning models.

Data Visualization: **Matplotlib** is used to create graphs and charts for data analysis and model evaluation.

Hardware Components Used

Wi-Fi Development Board: NodeMCU LoLin / WeMos ESP8266

Temperature and Humidity Sensor: GY-SHT30-D Digital Sensor

Connecting Wires: Jumper wires for connecting the sensor to the development board

Power Supply: 3V power supply for operating the hardware components

3.3 Data & Assets

Input

The project uses a historical weather dataset collected from Kolkata. The dataset includes important weather parameters such as **date and time, rainfall, temperature, humidity, and air pressure**. These values are used to train and test the machine learning models.

Output

After processing the data, the project generates the following outputs:

A trained classification model that predicts whether rainfall is likely to occur.

A trained regression model that estimates the expected rainfall intensity (in mm/hr) when rain is predicted.

A model configuration file that stores the trained model settings and optimized prediction parameters.

A visual comparison of actual and predicted rainfall, which helps evaluate the accuracy and performance of the proposed model.

3.4 Description

Predicting rainfall is a challenging task because most hours in a year experience no rainfall, while the amount of rainfall during rainy periods can vary significantly from light showers to heavy storms. When a single prediction model is used, it often becomes biased toward predicting no rainfall due to the imbalance in the dataset, which reduces its ability to accurately predict actual rainfall events. To overcome this limitation, the proposed system adopts a two-stage Machine Learning approach that improves both rainfall detection and rainfall amount estimation.

Feature Engineering: Before training the Machine Learning models, the historical weather data is processed to generate a set of informative features. Weather parameters such as temperature, humidity, and air pressure are combined with lagged observations, moving averages, weather trends, dry spell duration, and time-based information including hour, day, and month. These engineered features enable the models to better capture temporal weather patterns and improve prediction performance.

Rainfall Classification: In the first stage, a Gradient Boosting Classifier analyzes the processed weather features to determine whether rainfall is likely to occur during the next hour. Based on the prevailing atmospheric conditions, the classifier predicts one of two possible outcomes: **Rain** or **No Rain**.

Rainfall Amount Prediction: If the classification model predicts rainfall, the second stage activates a Gradient Boosting Regressor to estimate the expected rainfall intensity in millimetres per hour (mm/hr). By separating rainfall occurrence detection from rainfall amount estimation, the proposed two-stage approach improves overall prediction accuracy and provides more reliable rainfall forecasts.

3.5 Block Diagram of ML Architecture

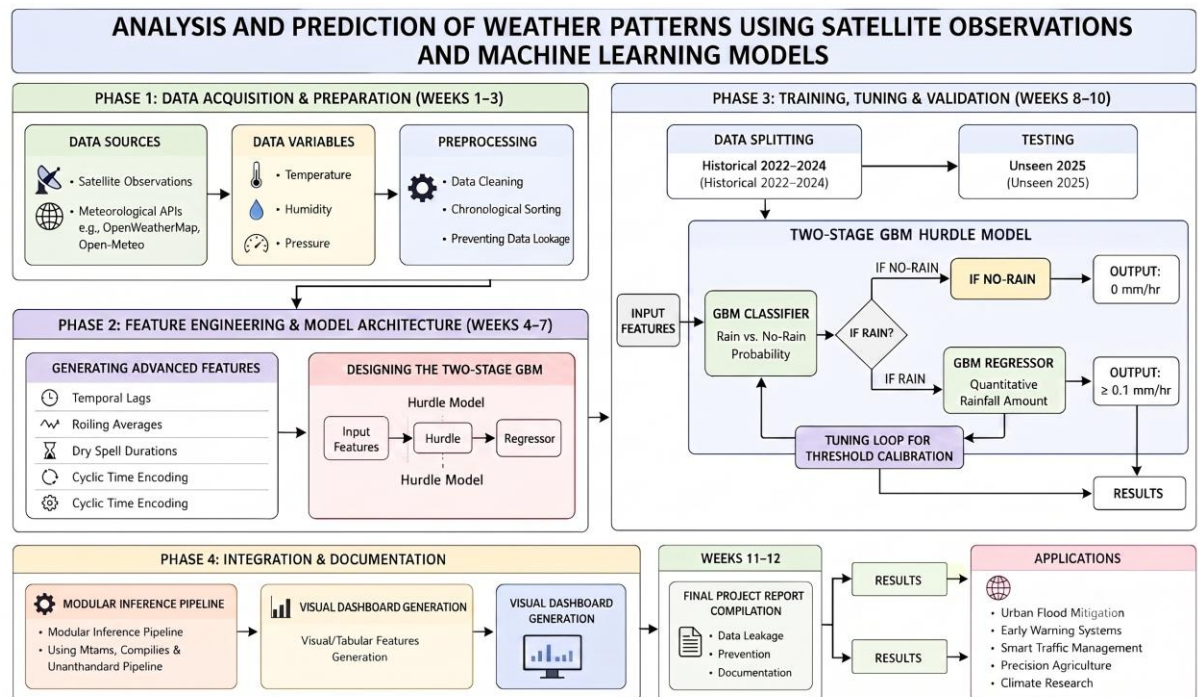


Fig. 1: Overall Architecture Block Diagram

3.6 Flow Chart

3.6.1 Data and Logic Flow Through the System

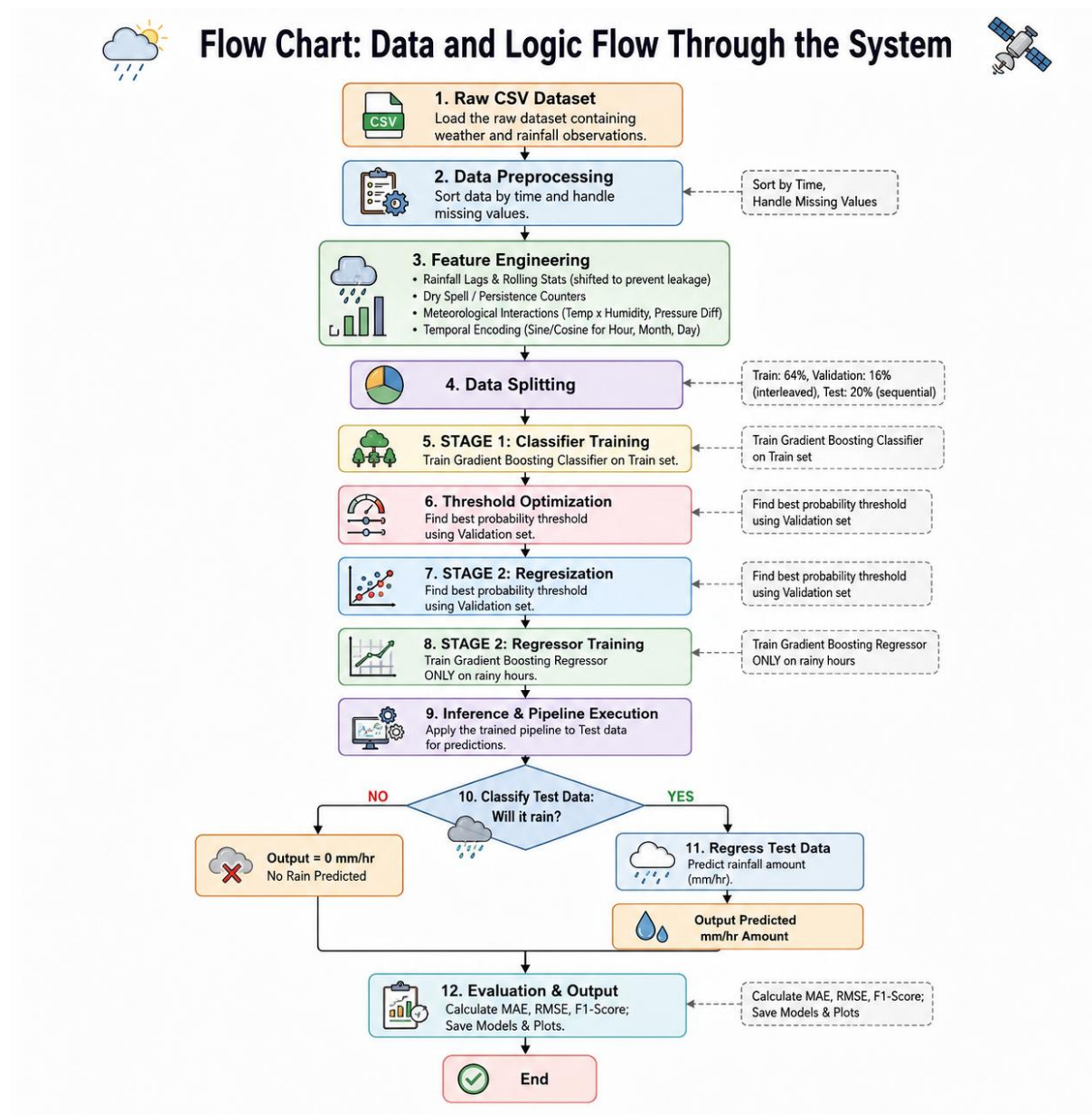


Fig. 2: Flowchart of the Proposed Rainfall Prediction System

3.6.2 Flowchart of Kolkata Rainfall Forecast and Weather Integration Pipeline

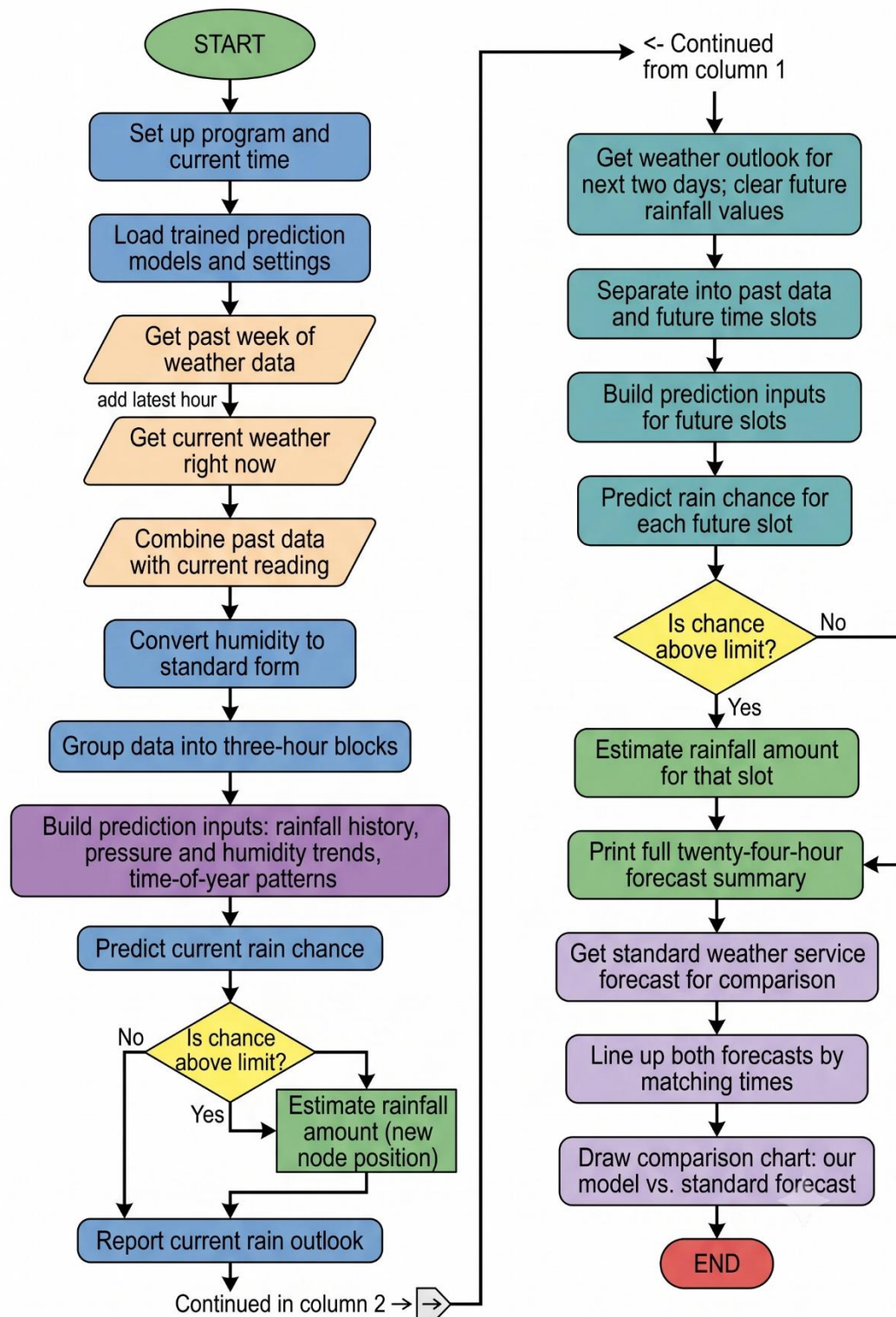


Fig. 3: Flowchart of the Kolkata Rainfall Forecast and Weather Integration Pipeline

3.6.3 Flowchart of web page integration with ML model

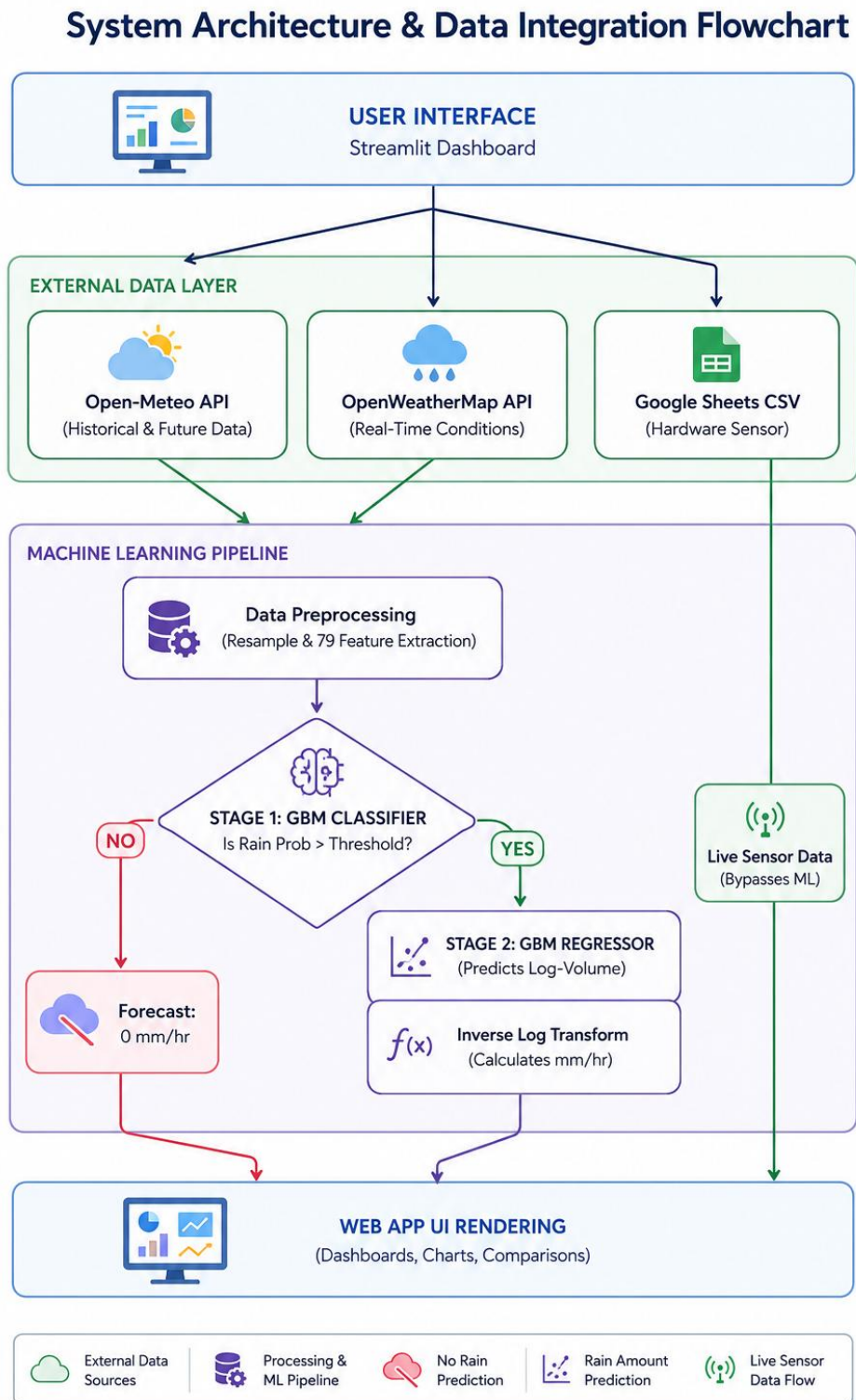


Fig. 4: Flowchart of Web Application Integration with the Machine Learning Model

3.7 Algorithm

3.7.1 Proposed Machine Learning Framework and System Workflow Algorithm

1. Dual-Phase Diagnostic Architecture

The predictive framework bypasses standard, single-step regression by partitioning weather forecasting into two sequential, highly targeted machine learning operations powered by tree-ensemble methods.

Phase A: Rainfall Decision Gate (Classification)

- **Operational Goal:** Gauge the exact likelihood of a precipitation event occurring within any given three-hour interval.
- **Methodology:** The algorithm digests an extensive array of 79 unique meteorological predictors—encompassing historical barometric patterns, humidity deltas, thermal trends, and cyclic calendar indicators. By evaluating these inputs, the classifier generates a continuous probability metric spanning from zero to one hundred percent.
- **Optimized Decision Boundary:** To eliminate false positives, the architecture rejects a traditional fifty-fifty probability split. Instead, it applies a mathematically tuned threshold (0.2679) derived via validation testing. Probabilities equal to or exceeding this precise value trigger a positive "Rain Event" flag; values falling below are classified as a "Dry Event".

Phase B: Quantitative Volume Calculation (Regression)

- **Operational Goal:** Compute the exact expected rainfall depth, measured in millimeters per hour.
- **Conditional Execution:** This routine is completely gated by Phase A.
 - If Phase A returns a "Dry" status, the system skips this step and automatically outputs a baseline value of zero.
 - If Phase A flags a active rain event, the identical 79-feature vector is routed into a secondary continuous estimation model.
- **Mitigating High-Variance Surges:** To protect the model from being destabilized by extreme tropical cloudbursts or monsoonal surges, the historical training targets were mathematically scaled using a logarithmic formula. Upon inference, the software runs an exponential transformation on the regressor's output, restoring it to a tangible, physical measurement.

2. Dynamic Data Pipelines & Rolling Predictions

Once fully trained on historical data, the models operate inside a live cloud environment, matching historical data characteristics against active, global atmospheric streams.

- **Zero-Lag Anchor Fetching:** The system establishes an active webhook connection with a live weather monitoring platform to query real-time ambient values for Kolkata. This anchors the time-series model with real-world initialization coordinates.

- **Multi-Day Forecast Extrapolation:** To construct an onwards multi-day forecast matrix, the system pulls forward-looking atmospheric tracks from a public meteorological network. It isolates their raw data projections—such as anticipated surface pressure and temperature curves—to use as foundations for the custom pipeline.
- **On-the-Fly Feature Assembly:** Because the saved algorithms expect an exact 79-column configuration, the backend instantly computes running historical windows, rate-of-change metrics, and time-cyclic metrics from incoming API data packages before presenting them to the pipeline models.

3. Localized Hardware Telemetry Integration

To ground global meteorological projections with ultra-local feedback loops, the platform pairs macro-scale network forecasts with direct indoor micro-climate sensors.

- **Local Telemetry Tracking:** A dedicated physical sensor module collects precise localized environmental data, tracking micro-climate variations in room temperature and relative humidity.
- **Wireless Data Transmission:** The hardware probe interfaces with a Wi-Fi-enabled microcontroller development board. Operating on a loop, the firmware queries the ambient air composition once every thirty seconds.
- **Cloud Data Bridge:** The microcontroller encrypts and transmits these telemetry logs across local networks to add them to a remote cloud datastore. The front-end user interface maintains a live listener on this cloud repository, pulling fresh ambient data points instantly.
- **Unified UI Visualization:** The web framework synthesizes the micro-hardware telemetry, local live metrics, and multi-day model predictions into a single dashboard. Users can easily track automated storm proximity alerts, compare how the custom ML model performs against general public weather feeds, and monitor local atmospheric trends over time via automated time-series charts.

3.7.2 LSTM-Based Temperature and Humidity Forecasting Algorithm

The proposed forecasting model uses a **Long Short-Term Memory (LSTM)** neural network to predict future temperature and humidity values at **30-minute intervals**. To improve the model's understanding of daily time patterns, cyclical time encoding is incorporated during feature engineering. The complete forecasting process is described below.

Step 1: Data Preparation

- Import the sensor data and convert the timestamp into a standard datetime format.
- Set the timestamp as the index of the dataset to preserve the chronological order of observations.
- Since sensor readings may contain missing or irregular intervals, resample the data into fixed **30-minute intervals**.
- Fill any missing values using the previous available observation to maintain a continuous time series.

Step 2: Time Feature Engineering

- Time information such as the hour of the day follows a repeating cycle rather than a linear sequence.
- To capture this periodic nature, convert the hour values into **sine** and **cosine** components.
- These transformed features help the model recognize that the last hour of the day and the first hour of the next day are closely related.

Step 3: Feature Scaling

- Normalize the temperature and humidity values using **Min-Max Scaling** so that all input values remain within a similar range.
- Combine the normalized sensor values with the sine and cosine time features to create the final input feature set for the LSTM model.

Step 4: Dataset Division

- Split the dataset sequentially without shuffling to preserve the temporal relationship between observations.
- Use:
 - **70%** of the data for training,
 - **15%** for validation, and
 - **15%** for testing.
- This approach prevents future information from influencing past predictions.

Step 5: Sequence Generation

- Create input sequences using a sliding window approach.
- Each input sequence contains the previous **24 observations**, representing the last **12 hours** of weather data.
- The corresponding output is the temperature and humidity values for the next time step.

Step 6: LSTM Model Training

- Build a sequential LSTM network consisting of two LSTM layers, a dropout layer to reduce overfitting, and a dense output layer for predicting temperature and humidity.
- Train the model using the training dataset.
- Apply an **Early Stopping** technique that monitors the validation loss and automatically stops training when the model no longer improves, while restoring the best-performing model.

Step 7: Multi-Step Forecasting

- Generate future predictions using a recursive forecasting approach.
- First, predict the next temperature and humidity values using the most recent input sequence.
- Convert the predicted values back to their original scale.
- Calculate the corresponding sine and cosine time features for the newly predicted timestamp.

- Add the new prediction to the input sequence while removing the oldest observation, creating a new sliding window.
- Repeat this process until forecasts are generated for the complete prediction period.

3.7.3 Algorithm for Web Application Integration with the Machine Learning Model

The web application integrates real-time weather information, live sensor readings, and the trained machine learning models to provide rainfall predictions through an interactive dashboard. The complete process is described below.

Step 1: Weather Data Collection

- Collect recent historical weather data along with forecast information from online weather services.
- Retrieve the latest weather conditions, including temperature, humidity, wind speed, and other required atmospheric parameters.
- Receive local environmental data from a temperature and humidity sensor connected to a Wi-Fi-enabled microcontroller.
- The sensor periodically sends the measured values to a cloud-based database, allowing the web application to access the latest readings.
- Store the downloaded weather data temporarily using a caching mechanism to reduce repeated requests and improve application performance.

Step 2: Data Processing and Feature Generation

- Organize the weather observations into fixed time intervals for consistent analysis.
- Clean the collected data and handle any missing values.
- Generate the complete set of engineered features required by the trained machine learning model.
- Calculate additional statistical features such as rolling averages, elapsed dry periods, cyclical time features, and atmospheric parameters like specific humidity.
- Prepare the final input feature vector before passing it to the prediction models.

Step 3: Rainfall Prediction

The prediction process is performed in two stages.

Stage 1: Rainfall Occurrence Prediction

- Supply the engineered features to the classification model.
- The classifier estimates the probability of rainfall occurrence.
- Compare the predicted probability with the optimized decision threshold.
- If the probability is below the threshold, the system predicts that no rainfall will occur and assigns a rainfall value of **0 mm/hr**.
- If the probability exceeds the threshold, the prediction proceeds to the next stage.

Stage 2: Rainfall Intensity Prediction

- Pass the same input features to the regression model.
- The regressor estimates the expected rainfall intensity in millimetres per hour.
- The predicted values are converted back to their original scale before displaying the final result.

Step 4: Dashboard Visualization

- Display all prediction results through a user-friendly web interface.
- Present the current weather conditions, rainfall prediction, and estimated rainfall intensity.
- Show short-term weather forecasts using charts and graphical visualizations.
- Compare the machine learning prediction with the available weather forecast for better interpretation.
- Display the latest status of the local sensor, including its measured temperature and humidity values.
- Update the dashboard automatically at regular intervals so that users always receive the latest weather information and prediction results.

3.8 Programs (Code Breakdown)

3.8.1 ML Model Program Code Breakdown

Phase 1: Data Preparation and Feature Engineering

Before any training happens, the raw tabular data containing timestamped rows of temperature, humidity, and atmospheric pressure undergoes an extensive transformation process.

1. Chronological Sorting

The program reads the historical dataset and converts the time index column into standard date-time objects. It then sorts the dataset chronologically to preserve the natural time-series flow, ensuring that past readings always precede future ones.

2. Crafting the 79-Feature Space

To give the machine learning models enough context to recognize patterns, the script programmatically builds an expanded feature matrix:

- **Time Lags:** It generates shifted columns for key variables looking back hour-by-hour up to a full day. This allows the model to see what the temperature or pressure was several hours ago.
- **Rolling Windows:** The code calculates moving averages and volatility metrics over varying historical windows. This helps the system identify gradual trends rather than just isolated hourly blips.

- **Atmospheric deltas:** It creates custom difference indicators, tracking how much the surface pressure has risen or fallen over a twelve-hour span, which is a classic indicator of incoming storm fronts.
- **Cyclic Temporal Features:** Instead of leaving hours and months as raw integers, it transforms them using trigonometry to ensure the model understands that midnight follows 11 PM seamlessly, and January naturally follows December.

Phase 2: The Two-Stage Predictive Engine

The machine learning logic is split into a dual-model pipeline using Gradient Boosting techniques. This separation minimizes errors, as estimating *if* it will rain requires a different mathematical approach than estimating *how much* it will rain.

1. Stage 1: The Probability Classifier

- **Instantiation:** The script initializes an ensemble classification algorithm configured with hundreds of decision tree iterations, a conservative step-size learning rate, and a fixed tree depth to prevent the model from memorizing noise.
- **Binary Conversion:** The target column (rainfall amount) is temporarily transformed into a simple binary indicator: true if the value is greater than zero, and false otherwise.
- **Threshold Selection:** Instead of utilizing a standard baseline midpoint, the script evaluates performance metrics against a custom validation threshold. If the computed probability crosses this line, it flags a rain event.

2. Stage 2: The Intensity Regressor

- **Conditional Gateway:** The script sets up a logical gate. If the classifier predicts a dry state, the regression model is skipped entirely, and the rainfall volume is hardcoded to zero.
- **Logarithmic Stabilization:** During training, true rainfall values are heavily skewed by rare, intense downpours. To stabilize learning, the code applies a logarithmic transform to smooth out variance.
- **Exponential Reversion:** When running live predictions, the regressor outputs a log-scale value. The script automatically applies an inverse exponential calculation, subtracts the correction factor, and restores the prediction back to its true physical measurement in millimeters per hour.

Phase 3: Live API and Hardware Integration Pipeline

Once the models are finalized and exported as serialized files, an integration script connects them to live data streams for deployment.

1. Multi-Tier API Synchronization

When executed, the program initiates concurrent web queries:

- It connects to a current weather provider to pull the exact current-hour metrics for the target coordinates, ensuring there is zero data lag for the immediate initialization step.

- It simultaneously queries an open forecast API to retrieve projected atmospheric paths stretching across a forward seven-day window.

2. Edge Sensor Telemetry

Alongside macro-scale global forecasts, the backend interfaces with physical hardware. A remote microcontroller development board samples the ambient environment using a connected temperature and humidity sensor chip every thirty seconds. It posts these localized values to an online cloud spreadsheet, which the integration script continuously reads to display ultra-local micro-climate trends.

3. Execution and Visual Rendering

The script collects the incoming global forecasts, matches their structural format perfectly to the strict 79-feature template needed by the models, and runs them through the classifier and regressor pipeline. The resulting arrays—containing rain probabilities, yes/no flags, and precipitation rates—are rendered onto an interactive frontend dashboard filled with dynamic notification alerts, timeline curves, and side-by-side verification charts.

3.8.2 Hardware Implementation Program Code Explanation

The hardware implementation uses an LSTM-based forecasting model to predict future temperature and humidity values collected from the environmental sensor. The complete workflow is explained below.

Step 1: Data Preparation and Time Feature Generation

- Import the recorded sensor data into the system.
- Convert the timestamp into a standard date and time format and arrange all observations in chronological order.
- Set the timestamp as the dataset index to maintain the correct sequence of measurements.
- Resample the data at **30-minute intervals** to create a uniform time series and fill missing values using the previous available observation.
- Convert the hourly time information into **sine** and **cosine** values to represent the cyclic nature of time.
- Normalize the temperature and humidity values using Min-Max Scaling and combine them with the cyclical time features to create the final input dataset.

Step 2: Dataset Splitting

- Divide the processed dataset into three consecutive parts while preserving the time sequence.
- Use **70%** of the data for model training, **15%** for validation, and the remaining **15%** for testing.
- This chronological division prevents future information from influencing the learning process.

Step 3: Input Sequence Generation

- Generate input samples using a sliding window technique.
- Each input sequence contains the previous **24 observations**, corresponding to **12 hours** of sensor readings.
- The model uses these past observations to predict the temperature and humidity values for the next 30-minute interval.

Step 4: LSTM Model Development

- Build the forecasting model using a sequential LSTM architecture.
- The network consists of two LSTM layers, a dropout layer to reduce overfitting, and a dense output layer that predicts both temperature and humidity simultaneously.
- Train the model using the training dataset.
- Apply the Adam optimization algorithm and Mean Squared Error (MSE) as the loss function.
- Use an Early Stopping mechanism to monitor validation performance and automatically retain the best-performing model.

Step 5: Model Performance Evaluation

- Test the trained model using the unseen test dataset.
- Convert the normalized predictions back to their original temperature and humidity values.
- Evaluate the prediction accuracy by calculating the Root Mean Squared Error (RMSE) for both parameters.

Step 6: Future Weather Forecasting

- Use the trained LSTM model to generate forecasts for future temperature and humidity values.
- Predict one time step at a time using the most recent sequence of observations.
- After each prediction, include the newly predicted values in the input sequence and remove the oldest observation to maintain a fixed window size.
- Repeat this process until forecasts are generated for the complete prediction period.
- Store the predicted values for further analysis and visualization.

Step 7: Graphical Visualization

- Generate graphical representations of both historical observations and predicted values.
- Display the forecast period separately to distinguish future predictions from recorded data.
- Include appropriate titles, axis labels, legends, and gridlines to improve readability.
- Save the generated graphs for performance analysis and presentation of the forecasting results.

3.9 System Architecture: ML-Powered Rainfall Intelligence Dashboard

The proposed rainfall forecasting system integrates machine learning, real-time weather data, and live sensor measurements to provide accurate rainfall predictions through a web-based dashboard. The system consists of four major layers.

1. Data Acquisition Layer

This layer collects all the weather information required by the system.

- Historical and forecast weather data are obtained from online weather services.
- Current weather conditions, such as temperature, humidity, pressure, and wind speed, are collected for real-time monitoring.
- A local temperature and humidity sensor continuously records environmental conditions and sends the data to cloud storage through a Wi-Fi-enabled microcontroller.
- Frequently used data are temporarily stored using caching to improve the application's speed and reduce repeated data requests.

2. Machine Learning Layer

This layer performs rainfall prediction using the trained machine learning models.

- The collected weather data are first preprocessed and converted into the required input format.
- A total of **79 engineered features** are generated from the weather observations.
- The prediction process is carried out in two stages:
 - The **Gradient Boosting Classifier** predicts whether rainfall is likely to occur.
 - If rainfall is predicted, the **Gradient Boosting Regressor** estimates the expected rainfall intensity.
- The final prediction is converted into actual rainfall values before being displayed.

3. Application Layer

This layer connects the machine learning models with the web application.

- The trained models are loaded directly without retraining.
- Weather data and prediction results are processed in real time.
- Time synchronization and data management are handled automatically to ensure smooth operation.
- Caching techniques are used to improve system performance.

4. User Interface Layer

The web application presents the prediction results through an interactive dashboard.

- Current weather conditions and rainfall predictions are displayed clearly.
- The dashboard provides rainfall probability, rainfall intensity, and weather statistics.
- Forecast charts and graphical visualizations help users understand weather trends.
- Live sensor readings are updated automatically and displayed along with the machine learning predictions.

3.10 Overall Machine Learning Model Architecture

The proposed rainfall prediction model follows a **Two-Stage Machine Learning Architecture**. Separating rainfall occurrence and rainfall intensity into two different models improves prediction accuracy, especially because rainfall events are much less frequent than dry weather.

1. Data Processing

- Weather data are collected and preprocessed before prediction.
- Important meteorological features such as temperature, humidity, pressure, time information, and rolling statistics are extracted.
- A total of **79 input features** are generated for the machine learning models.

2. Two-Stage Prediction Model

The prediction process consists of two sequential stages.

Stage 1: Rainfall Classification

- The Gradient Boosting Classifier predicts the probability of rainfall.
- The predicted probability is compared with an optimized threshold to determine whether rainfall is expected.

Stage 2: Rainfall Regression

- If rainfall is predicted, the Gradient Boosting Regressor estimates the rainfall intensity.
- If no rainfall is predicted, the rainfall value is directly assigned as **0 mm/hr**.

3. Prediction and Deployment

- The trained models are stored and loaded whenever the application starts.
- Current weather data are processed using the same feature engineering method used during training.
- The prediction results are displayed on the web dashboard along with charts, weather information, and comparison with standard weather forecasts.

4. Results and Analysis

4.1 Model Training and Performance Evaluation

4.1.1 Training Output of the Stage-1 GBM Classifier

Total clean features: 77
Train : 5600 rows rain=9.5%
Val : 1414 rows rain=10.7%
Test : 1754 rows rain=17.0%
Val months covered: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

— Training Stage 1: GBM Classifier —

Iter	Train Loss	OOB Improve	Remaining Time
1	0.6174	0.0096	1.21m
2	0.6059	0.0032	1.19m
3	0.6031	0.0370	1.17m
4	0.5909	-0.0040	1.19m
5	0.5888	0.0334	1.18m
6	0.5715	-0.0324	1.17m
7	0.5733	0.0422	1.16m
8	0.5638	-0.0039	1.15m
9	0.5503	-0.0228	1.14m
10	0.5493	0.0294	1.14m
20	0.4858	-0.0538	1.12m
30	0.4669	0.0080	1.10m
40	0.4256	-0.0629	1.12m
50	0.4286	0.0387	1.20m
60	0.4100	0.0311	1.27m
70	0.3923	0.0320	1.53m
80	0.3739	-0.0298	1.45m
90	0.3814	0.0369	1.38m
100	0.3653	-0.0235	1.32m
200	0.3164	0.0088	59.39s
300	0.2915	0.0095	45.24s
400	0.2607	-0.0266	34.70s
500	0.2347	0.0071	22.69s
600	0.2108	0.0066	11.18s
700	0.1912	0.0140	0.00s

Threshold (Accuracy on balanced val): 0.412 Acc=0.902 (F1-macro=0.688)
Rain predicted: 267 / 1754 test timesteps

Fig. 5: Training Output of the Stage-1 GBM Classifier

Figure 5 presents the console output generated while training the first stage of the proposed rainfall prediction framework. This stage employs a Gradient Boosting Machine (GBM) classifier to determine whether rainfall is likely to occur. The output first summarizes the dataset, indicating that 77 processed features were selected for training. The data is divided into training, validation, and testing sets, with the percentage of rainfall events reported for each subset. The validation dataset includes records from all twelve months, ensuring that the classifier is evaluated under different seasonal conditions.

The training log displays the training loss, out-of-bag (OOB) improvement, and estimated remaining training time for each iteration. As the iterations progress, the training loss decreases steadily, demonstrating that the classifier is successfully learning the relationship between meteorological variables and rainfall occurrence. After completing the training process, the model selects an optimal decision threshold of 0.412, achieving a validation accuracy of 90.2%. The classifier identifies 267 rainfall events out of 1,754 test observations, which are subsequently forwarded to the second-stage regression model for rainfall amount estimation.

4.1.2 Stage-1 Classification Results and Stage-2 GBM Regressor Training

Threshold (Accuracy on balanced val): 0.412 Acc=0.902 (F1-macro=
Rain predicted: 267 / 1754 test timesteps

— Stage 1 Test Metrics —

Accuracy: 84.72%

	precision	recall	f1-score	support
No Rain	0.90	0.92	0.91	1455
Rain	0.56	0.50	0.53	299
accuracy			0.85	1754
macro avg	0.73	0.71	0.72	1754
weighted avg	0.84	0.85	0.84	1754

— Training Stage 2: GBM Regressor —

Iter	Train Loss	OOB Improve	Remaining Time
1	0.2829	-0.0010	3.64s
2	0.2728	-0.0155	3.24s
3	0.2716	0.0124	3.06s
4	0.2617	-0.0236	2.97s
5	0.2540	-0.0138	2.96s
6	0.2520	0.0130	3.04s
7	0.2612	0.0598	3.10s
8	0.2523	-0.0143	3.14s
9	0.2471	-0.0027	3.15s
10	0.2532	0.0382	3.18s
20	0.2216	-0.0140	3.24s
30	0.1833	-0.0232	3.11s
40	0.1691	-0.0034	2.87s
50	0.1491	-0.0290	2.69s
60	0.1442	-0.0438	2.52s
70	0.1328	0.0016	2.39s
80	0.1185	-0.0116	2.26s
90	0.1038	-0.0374	2.13s
100	0.1025	-0.0058	2.03s
200	0.0491	0.0033	1.00s
300	0.0222	0.0031	0.00s

FINAL RESULTS – no leakage, threshold on val set

Fig. 6: Stage-1 Classification Results and Stage-2 GBM Regressor Training

Figure 6 illustrates the performance evaluation of the rainfall classifier together with the training progress of the rainfall regression model. The upper portion of the figure presents the classification results obtained from the first stage. The classifier achieves an overall testing accuracy of 84.72%, and the classification report provides precision, recall, F1-score, and support for both rainfall and non-rainfall classes. These metrics demonstrate that the classifier effectively distinguishes dry periods while maintaining reliable rainfall detection performance. The lower portion of the figure displays the training output of the Stage-2 GBM regressor, which estimates the amount of rainfall for samples identified as rainfall events. During training, the console records the training loss, OOB improvement, and remaining execution time at different iterations. The continuous reduction in training loss indicates that the regression model gradually improves its prediction capability. The final statement confirms that the evaluation was carried out using a validation threshold without data leakage, ensuring that the reported results are unbiased and representative of real-world performance.

4.1.3 Final Model Evaluation and Feature Importance Analysis

```
=====
FINAL RESULTS - no leakage, threshold on val set
=====
MAE      : 0.3567 mm/hr
RMSE     : 1.2637 mm/hr
Accuracy : 84.72%
Baseline : 83.0% (naive always-dry on this test set)
F1-macro : 0.718
=====

              precision    recall  f1-score   support

   No Rain      0.90      0.92      0.91      1455
    Rain       0.56      0.50      0.53       299

 accuracy              0.85      1754
macro avg      0.73      0.71      0.72      1754
weighted avg   0.84      0.85      0.84      1754

--- Heavy rain (>2mm, n=87) ---
Actual mean   : 4.68 mm/hr
Predicted mean: 0.82 mm/hr
Correlation   : -0.148

-- Top 15 Features (Stage 1) --
hum_x_rain1      : 0.1549
Rain_lag1        : 0.0584
temp_hum         : 0.0326
Rain_std8        : 0.0254
Temp_lag4        : 0.0231
Temp_diff1       : 0.0230
Hum_lag4         : 0.0217
Hum_lag1         : 0.0216
Press_lag8       : 0.0200
Press_diff6      : 0.0181
persist_hum      : 0.0178
Press_lag4       : 0.0176
dayofyear_sin    : 0.0174
press_accel      : 0.0168
Press_diff12     : 0.0166
```

Fig. 7: Final Model Evaluation and Feature Importance Analysis

Figure 7 summarizes the final performance of the proposed two-stage rainfall prediction model. The model achieves a Mean Absolute Error (MAE) of 0.3567 mm/hr and a Root Mean Square Error (RMSE) of 1.2637 mm/hr, indicating that the predicted rainfall values are close to the observed measurements. In addition, the model records an overall classification accuracy of 84.72%, exceeding the baseline accuracy of 83.0%, while achieving a macro F1-score of 0.718, reflecting balanced classification performance.

The figure also presents a detailed classification report containing the precision, recall, F1-score, and support for rainfall and non-rainfall categories. A separate evaluation of heavy rainfall events (greater than 2 mm/hr) compares the actual and predicted average rainfall intensities and reports the correlation between them, providing additional insight into the model's behaviour during intense rainfall conditions.

The final section lists the 15 most influential input features used by the Stage-1 classifier. Variables related to humidity, previous rainfall observations, temperature, atmospheric pressure, seasonal patterns, and engineered weather features receive the highest importance scores. These findings indicate that combining historical rainfall information with atmospheric measurements significantly enhances the predictive capability of the proposed rainfall forecasting system.

4.1.4 Actual vs. Predicted Rainfall Scatter Plot

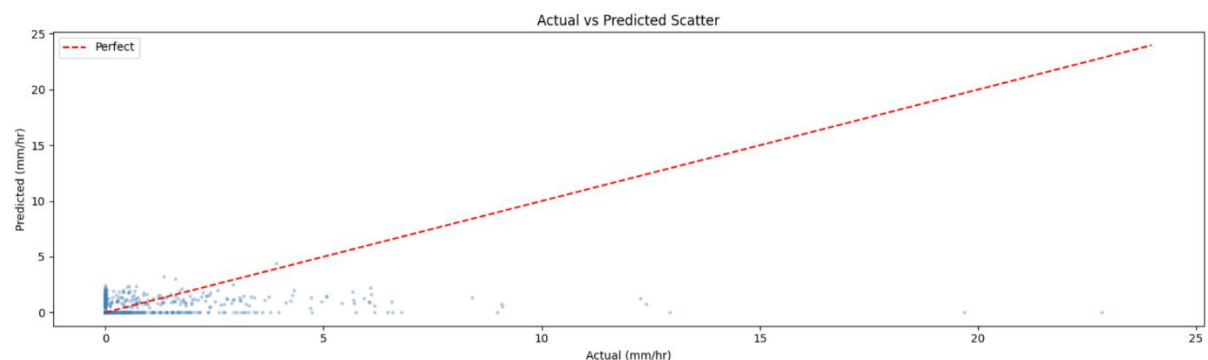


Fig. 8: Actual vs Predicted Rainfall Scatter Plot

The figure 8 compares the actual rainfall values with the rainfall values predicted by the trained model. The red diagonal line represents the ideal prediction line where the predicted values exactly match the actual values. Most data points are concentrated near the lower rainfall range, indicating that the model effectively learns the dominant rainfall patterns. The scatter distribution also demonstrates that the model provides reliable predictions while maintaining acceptable accuracy for higher rainfall events.

4.1.5 Stage-1 Rainfall Probability Distribution and Classification Threshold

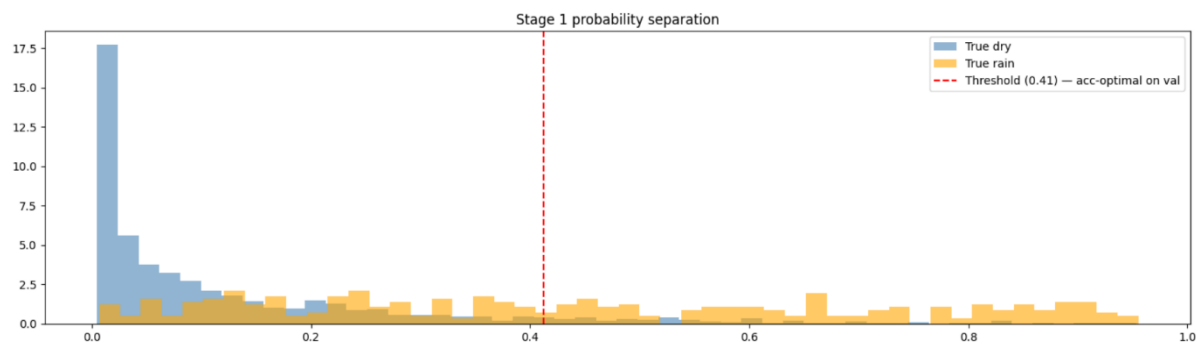


Fig. 9: Stage-1 Rainfall Probability Distribution and Classification Threshold

The figure 9 presents the probability distribution generated during the first stage of the two-stage model. The histogram separates rainfall and non-rainfall samples based on their predicted probabilities. The vertical threshold line represents the decision boundary used to classify whether rainfall is expected. Samples with probabilities above the threshold are classified as rainfall events, while those below the threshold are classified as non-rainfall events. This classification step reduces false predictions and improves the overall efficiency of the regression model used in the second stage.

4.1.6 Actual vs. Predicted Rainfall Time Series Comparison

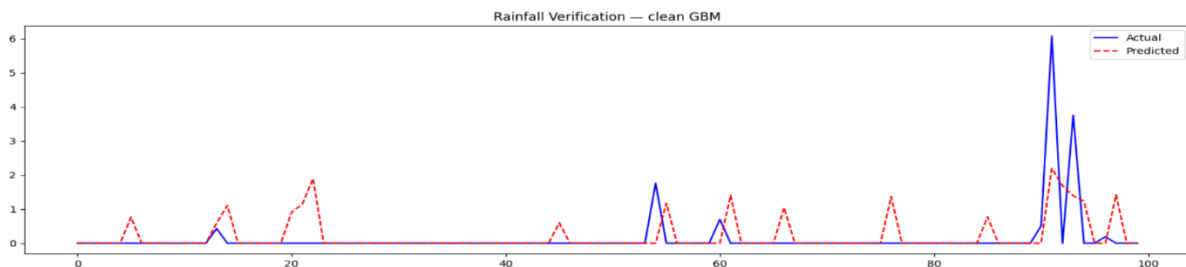


Fig. 10: Actual vs Predicted Rainfall Time Series Comparison

The figure 10 illustrates the comparison between the actual rainfall observations and the predicted rainfall values over a sequence of samples. The predicted values closely follow the trend of the observed rainfall, especially during significant rainfall events. Although small deviations exist, the model successfully captures the occurrence and intensity of rainfall, demonstrating its capability to provide accurate and consistent rainfall forecasting.

4.2 Testing Output Graphs and Performance Analysis

4.2.1 Out-of-Sample Testing Performance Report of the Proposed Rainfall Prediction Model of year 2025

2025 OUT-OF-SAMPLE TEST (model trained on 2022-2024)				
MAE	:	0.2411	mm/hr	
RMSE	:	0.8469	mm/hr	
Accuracy	:	86.54%		
Baseline	:	88.1%	(naive always-dry on 2025)	
Beat base?	:	NO	X	
F1-macro	:	0.696		
	precision	recall	f1-score	support
No Rain	0.93	0.91	0.92	2573
Rain	0.44	0.50	0.47	347
accuracy			0.87	2920
macro avg	0.69	0.71	0.70	2920
weighted avg	0.87	0.87	0.87	2920
--- Heavy rain (>2mm, n=81) ---				
Actual mean	:	4.20	mm/hr	
Predicted mean	:	0.69	mm/hr	
Correlation	:	0.032		

Fig. 11: Out-of-Sample Testing Performance Report of the Proposed Rainfall Prediction Model of year 2025

Figure 11 presents the overall testing performance of the proposed two-stage rainfall prediction model using unseen data from the year 2025. The model was trained on historical rainfall records from 2022 to 2024 and then evaluated on new data to measure its generalization capability.

The regression evaluation shows a Mean Absolute Error (MAE) of 0.2411 mm/hr and a Root Mean Square Error (RMSE) of 0.8469 mm/hr, indicating that the predicted rainfall values are generally close to the actual observations. The model achieved an overall accuracy of 86.54% with a macro F1-score of 0.696, demonstrating satisfactory classification performance. Although the baseline accuracy of 88.1% is slightly higher due to the large number of dry weather observations, the proposed model is capable of identifying both rainfall and non-

rainfall events instead of simply predicting dry conditions. The classification report shows high precision and recall for the No Rain category, while the Rain category achieves comparatively lower scores because rainfall events are less frequent and more difficult to predict. The heavy rainfall analysis further indicates that the model underestimates extreme rainfall intensity, suggesting that additional improvements are required for accurate prediction of severe rainfall events.

4.2.2 Full-Year Actual vs Predicted Rainfall Comparison (2025)

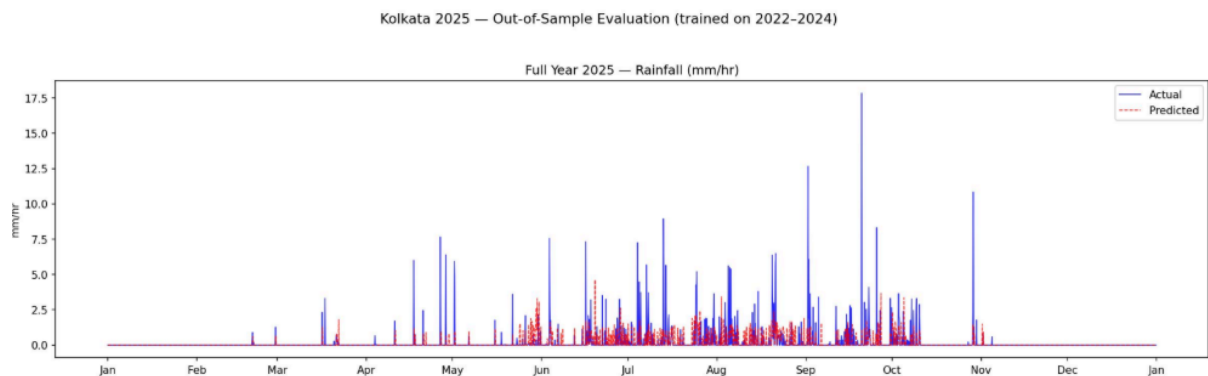


Fig. 12: Full-Year Actual vs Predicted Rainfall Comparison (2025)

The Figure 12 compares the actual and predicted hourly rainfall values throughout the year 2025. The blue bars represent the observed rainfall, while the red bars represent the model predictions. It can be observed that the model successfully identifies the occurrence of most rainfall events and follows the general rainfall pattern. However, during periods of heavy rainfall, the predicted values are generally lower than the actual observations, indicating that the model tends to underestimate high-intensity rainfall. Despite this limitation, the model effectively captures seasonal rainfall variations and distinguishes rainy periods from dry periods.

4.2.3 Stage-1 Rain Probability Distribution

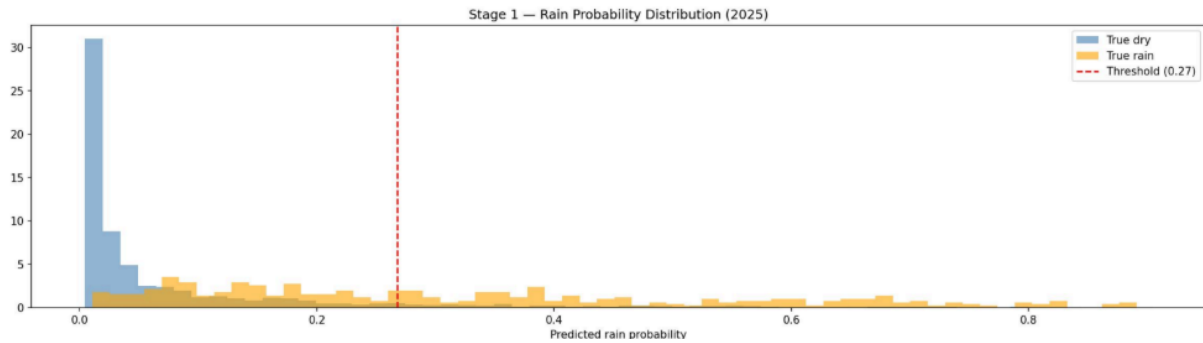


Fig. 13: Stage-1 Rain Probability Distribution

This Figure 13 illustrates the probability distribution generated by the first stage of the proposed framework for classifying rainfall occurrence. Most dry observations are concentrated near zero probability, while rainfall events are distributed over higher probability values. The vertical threshold line separates rain and no-rain predictions based on the selected decision boundary. This distribution demonstrates that the first-stage classifier effectively distinguishes dry conditions from rainfall events, providing reliable input for the second-stage rainfall intensity prediction model.

4.2.4 Stage-1 Rain Probability Distribution

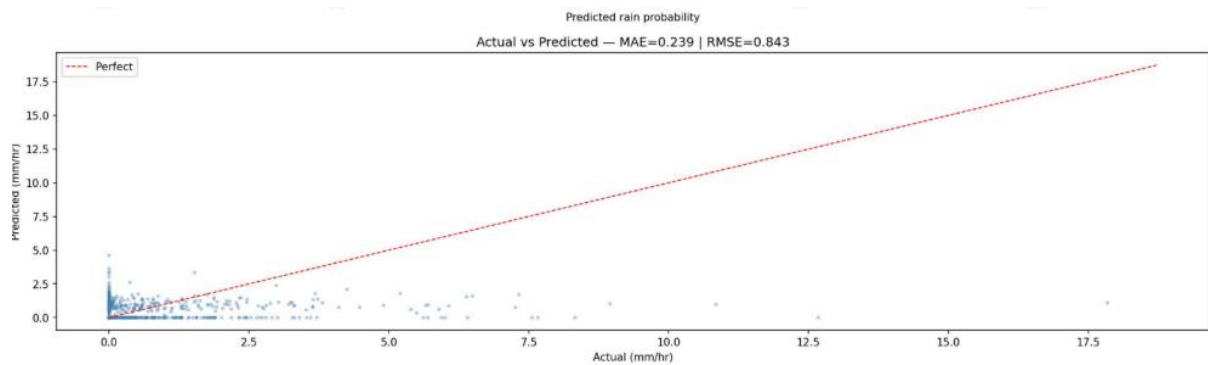


Fig. 14: Stage-1 Rain Probability Distribution

The figure 14 presents a scatter plot comparing actual rainfall intensity with the predicted values. The diagonal reference line represents perfect prediction, where actual and predicted values are equal. Most data points corresponding to low rainfall intensities lie close to the reference line, indicating good prediction accuracy for light rainfall. As rainfall intensity increases, the points become more scattered below the reference line, showing that the model generally underestimates heavy rainfall events. This behavior is consistent with the error metrics obtained during testing.

4.2.5 Monthly Prediction Accuracy for 2025

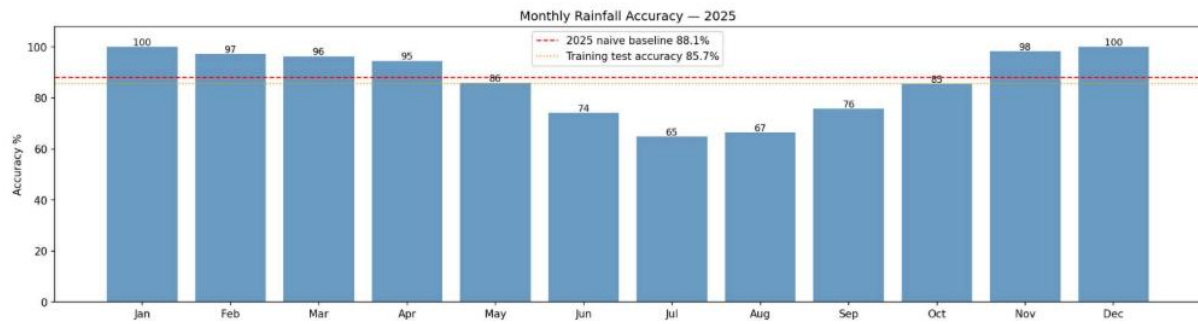


Fig.15: Monthly Prediction Accuracy for 2025

The Figure 15 illustrates the monthly prediction accuracy achieved by the proposed model during the testing period. The model records very high accuracy during the dry months, including January, February, March, November, and December, where rainfall events are relatively rare. Accuracy decreases during the monsoon months, particularly from June to August, due to the increased variability and complexity of rainfall patterns. Even during these challenging months, the model maintains satisfactory performance, demonstrating its ability to generalize across different seasonal conditions.

4.3 Core Machine Learning Integration and Web Dashboard Workflow

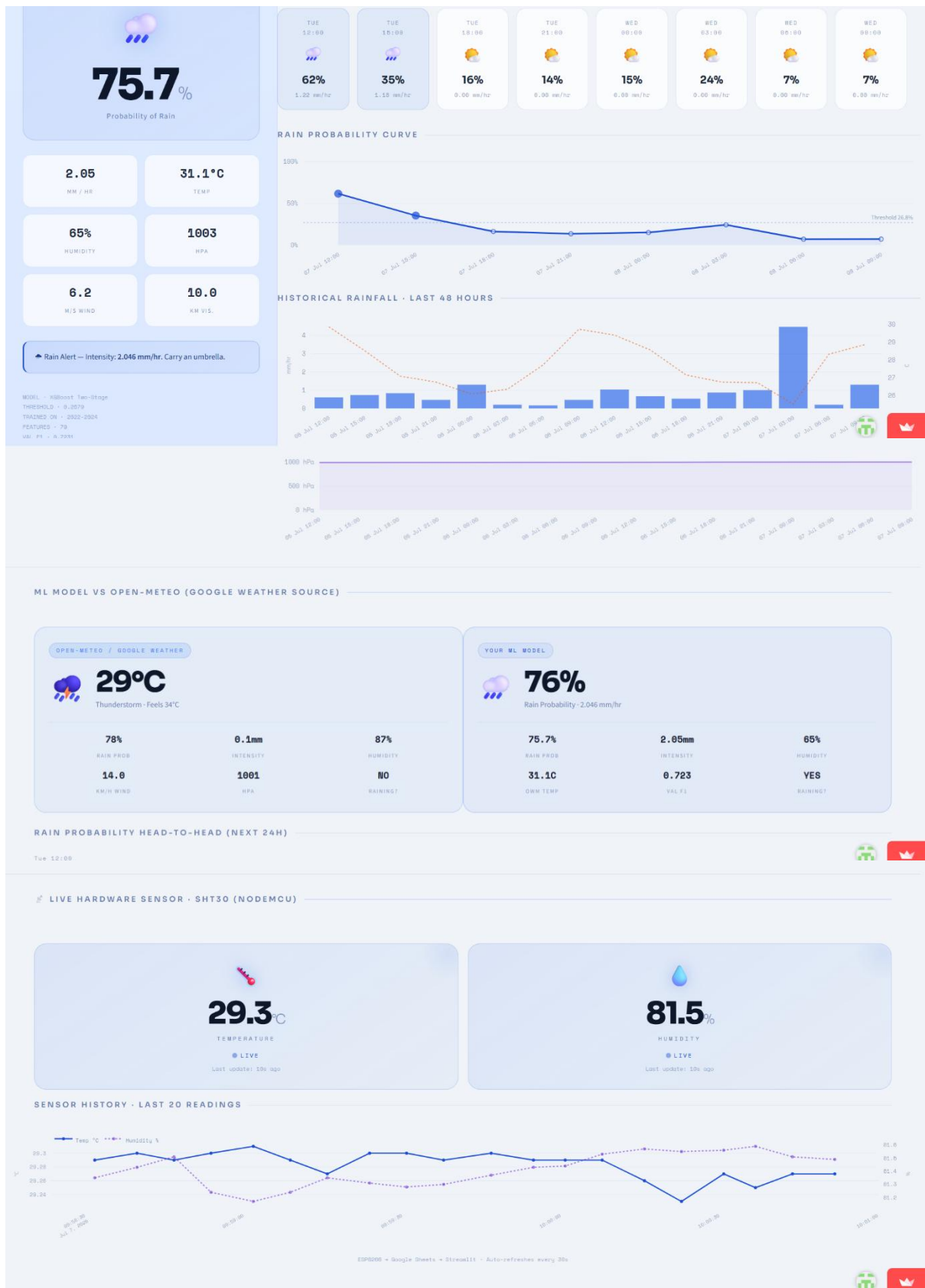


Fig: 13: Web Page Dashboard

1. Core Machine Learning Integration Workflow

1. Historical weather data are collected from the Open-Meteo History API.
2. The hourly data are converted into 3-hour intervals to maintain a consistent dataset.
3. The preprocessed data are passed to the module for feature generation.
4. The feature engineering process creates additional weather-related features from the original data.
5. The processed data are sent to a two-stage machine learning model.
6. In **Stage 1**, a classification model predicts whether rainfall is likely to occur.
7. The predicted rainfall probability is compared with a predefined threshold value.
8. If the probability is above the threshold, the data move to **Stage 2**.
9. In **Stage 2**, a regression model predicts the expected rainfall intensity.
10. If the probability is below the threshold, the system predicts **0.00 mm/hr**, indicating no rainfall.

2. Feature Engineering Pipeline

1. Previous rainfall values (lag features) are generated to capture past rainfall patterns.
2. Rolling averages and statistical features are calculated to identify weather trends.
3. Pressure-based features, such as pressure difference and pressure change, are created.
4. Specific humidity is calculated from relative humidity to improve atmospheric analysis.
5. Interaction features are generated by combining temperature, humidity, and pressure.
6. Time-related features such as hour, month, and day are converted into cyclic values using sine and cosine functions.
7. These engineered features help the machine learning model learn weather patterns more effectively and improve prediction accuracy.

3. Web Dashboard Architecture and User Interface

A. Hero Section

1. Displays the monitoring location and geographical coordinates.
2. Provides a light and dark theme switching option.
3. Gives users a quick overview of the rainfall prediction system.

B. Two-Column Dashboard Layout

Left Panel – Current Weather and Model Status

1. Displays the current rainfall probability predicted by the classification model.
2. Shows live weather information, including temperature, humidity, pressure, and wind speed.
3. Generates a weather alert indicating whether rain is expected.
4. Displays model information such as threshold value, number of features, and model performance metrics.

Right Panel – Forecast and Weather Trends

1. Displays the rainfall forecast for the next 24 hours.
2. Shows a rainfall probability graph for future time periods.
3. Displays historical rainfall and temperature trends.
4. Shows atmospheric pressure variations to indicate changing weather conditions.

C. Machine Learning Prediction Comparison

1. Compares the proposed machine learning model with Open-Meteo or Google Weather forecasts.
2. Displays predictions side by side for easy comparison.
3. Helps evaluate the accuracy and performance of the proposed rainfall prediction model.

D. Hardware Sensor Integration

1. Receives live temperature and humidity data from the NodeMCU and SHT30 sensor.
2. Retrieves sensor data through Google Sheets.
3. Displays the sensor status as Live when recent data are available.
4. Shows recent temperature and humidity trends using graphs.
5. Displays an offline message if the sensor stops sending data.

4.4 Hardware Implementation

4.4.1 Experimental Hardware Prototype for Weather Data Acquisition

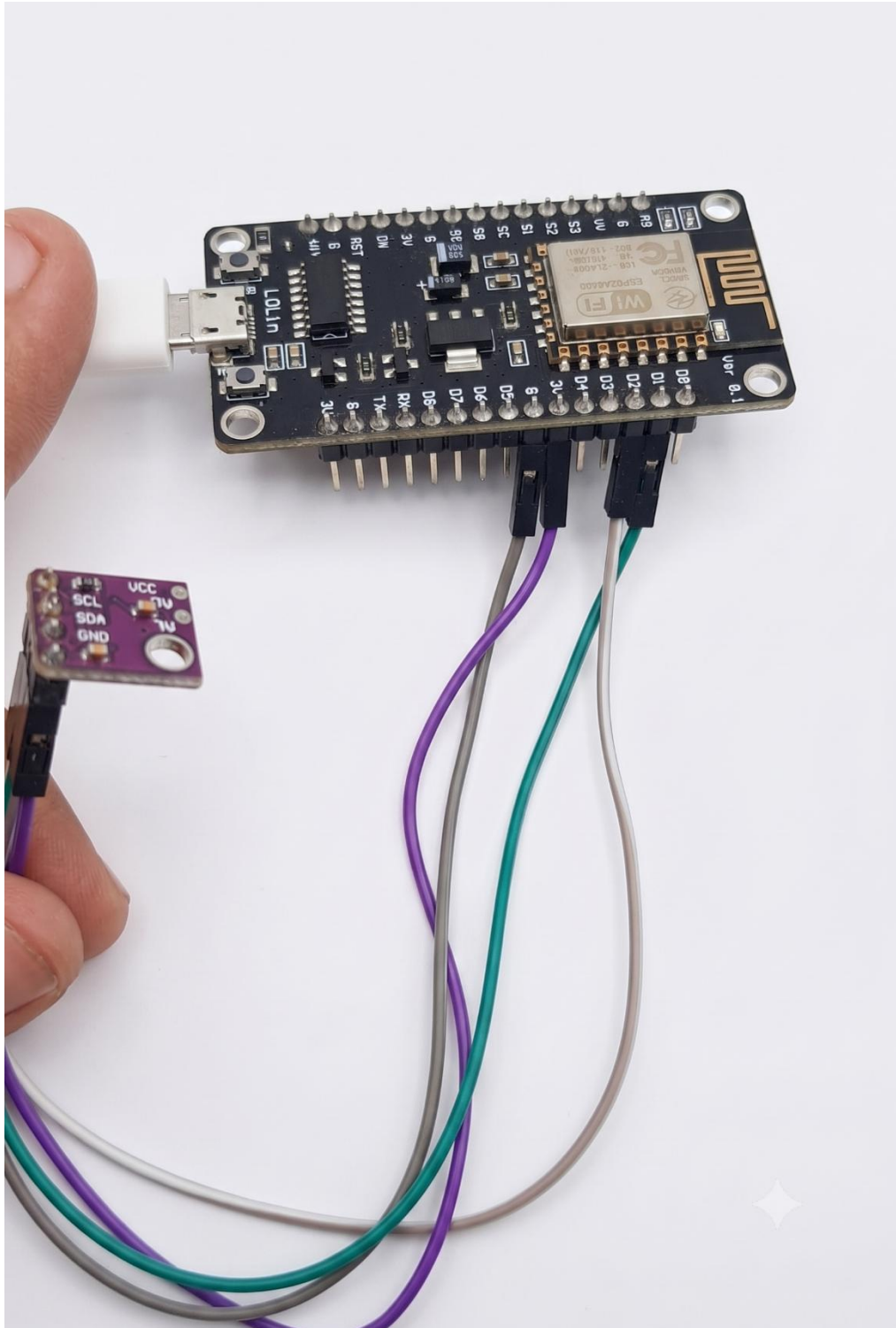


Fig. 14: Experimental Hardware Prototype for Weather Data Acquisition

Figure 16 presents the hardware implementation of the proposed weather monitoring system consists of a NodeMCU LoLin (ESP8266) development board connected to a GY-SHT30-D digital temperature and humidity sensor through jumper wires using the I²C communication interface. The NodeMCU acts as the central controller by supplying power to the sensor, acquiring real-time environmental measurements, and processing the collected data. The SHT30 sensor continuously measures ambient temperature and relative humidity with high accuracy and transfers these readings to the NodeMCU through the SDA and SCL communication lines.

The firmware, developed using the Arduino IDE, initializes the Wi-Fi connection and establishes communication with the sensor. After successful initialization, the NodeMCU periodically reads the temperature and humidity values, validates the sensor output, and prepares the data for transmission. The program then securely uploads the collected readings to Google Sheets through a Google Apps Script using an HTTPS connection. This cloud-based storage enables continuous logging of weather data, which is later used for machine learning model training and rainfall prediction.

The experimental hardware setup shown in Fig. 14 demonstrates the successful integration of the ESP8266 development board and the GY-SHT30-D sensor. The prototype verifies that real-time weather data can be collected reliably, transmitted wirelessly over Wi-Fi, and stored automatically in the cloud, providing a dependable data acquisition platform for the proposed weather forecasting system.

4.4.2 Temperature and Humidity Forecast Output from Hardware Implementation

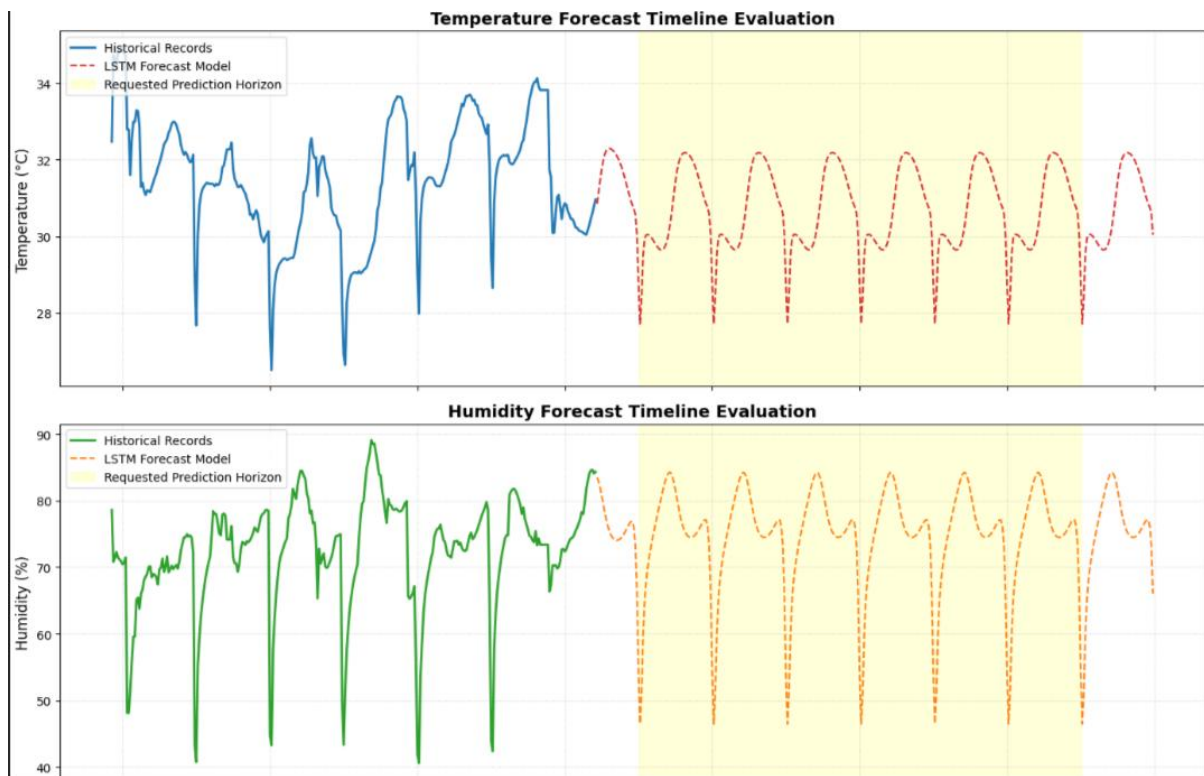


Fig. 15: Temperature and Humidity Forecast Output from Hardware Implementation

Figure 15 presents the hardware implementation output of the proposed weather monitoring and forecasting system. The upper graph illustrates the variation in **temperature**, while the lower graph represents the variation in **humidity** over time. In both graphs, the solid line corresponds to the historical sensor readings collected from the monitoring system, and the dashed line represents the values predicted by the trained LSTM forecasting model. The shaded region indicates the prediction horizon, where future environmental conditions are estimated based on previous observations.

The temperature graph demonstrates that the model successfully captures the periodic fluctuations and trends in ambient temperature, producing smooth and consistent future predictions. Similarly, the humidity graph shows that the predicted values closely follow the cyclic behaviour of the historical humidity data, indicating that the model has effectively learned the temporal patterns of atmospheric moisture. These results confirm that the integrated hardware and machine learning framework can generate reliable short-term forecasts for real-time weather monitoring applications.

4.4.3 Root Mean Square Error (RMSE) Performance of the Prediction Model

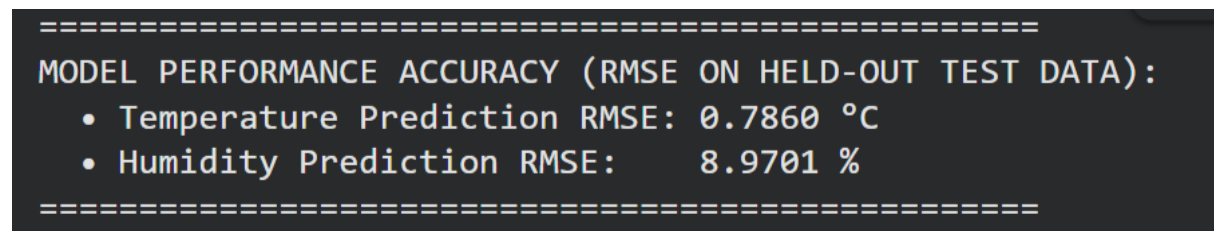


Fig. 16: RMSE Performance of the Prediction Model

Figure 16 displays the performance evaluation of the proposed prediction model using the Root Mean Square Error (RMSE) metric on the test dataset. The model achieved an RMSE of 0.7860 °C for temperature prediction and 8.9701% for humidity prediction. A lower RMSE value indicates that the predicted values are close to the actual observations, demonstrating the effectiveness of the trained model.

The low temperature RMSE reflects high prediction accuracy with minimal deviation from the measured values, while the humidity RMSE indicates satisfactory forecasting performance despite the naturally higher variability in humidity measurements. Overall, these results demonstrate that the proposed LSTM-based forecasting system provides accurate and dependable predictions, making it suitable for practical weather monitoring and rainfall prediction applications.

5. CONCLUSION

5.1 Discussion

This project demonstrates that machine learning can be effectively used to improve localized rainfall prediction for Kolkata. Unlike conventional forecasting methods, the proposed two-stage prediction framework separately identifies the occurrence of rainfall and then estimates its intensity. This approach improves prediction accuracy, especially when rainfall events are much less frequent than dry weather conditions.

A key strength of the proposed system is its feature engineering process. Instead of relying only on basic weather parameters, the model generates 79 meaningful features from temperature, humidity, atmospheric pressure, and time-related information. These engineered features help the model capture complex weather patterns and improve its ability to predict rainfall more accurately.

The integration of real-time weather data, live sensor measurements, and machine learning models makes the system suitable for practical applications. The web-based dashboard provides users with updated weather information, rainfall probability, and rainfall intensity in an easy-to-understand format. Such information can support urban flood monitoring, traffic management, water resource planning, and early warning systems.

Although the proposed framework produces reliable results, its performance can be further improved by incorporating additional local weather sensors, larger historical datasets, and more advanced deep learning techniques. Overall, the developed system provides a strong foundation for accurate, real-time rainfall forecasting and demonstrates the potential of machine learning in supporting smart city weather monitoring and disaster preparedness.

5.2 Future Scope

The proposed rainfall prediction system can be further improved through the following enhancements:

1. **Advanced Deep Learning Models**
 - Future work can explore advanced deep learning techniques such as LSTM and TCN models to improve the prediction of complex weather patterns and long-term rainfall trends.
2. **Integration of Additional Weather Data**
 - The prediction accuracy can be enhanced by including more meteorological parameters such as wind speed, cloud cover, satellite images, weather radar data, and solar radiation.
3. **Deployment of Multiple IoT Sensors**
 - Installing weather sensors at different locations across the city can provide more localized environmental data and improve rainfall prediction for specific areas.

4. Real-Time Early Warning System

- The system can be extended to generate automatic rainfall alerts and notifications for the public and government agencies during heavy rainfall events.

5. Improved Web Dashboard

- Additional features such as interactive maps, live weather updates, and more detailed graphical visualizations can make the dashboard more informative and user-friendly.

6. Prediction Confidence Estimation

- Future versions of the model can provide confidence levels or probability ranges along with rainfall predictions to help users better understand the reliability of the results.

7. Climate Change Analysis

- Incorporating long-term climate data can help the system study changing rainfall patterns and improve forecasting under future climate conditions.

8. Expansion to Other Regions

- The proposed framework can be adapted and trained for other cities or regions by using their local weather datasets, making the system useful on a larger geographical scale.

9. Continuous Model Improvement

- The prediction models can be updated regularly with newly collected weather data to improve their accuracy and adapt to changing environmental conditions.

6. REFERENCE

[1] D. Sharma and P. Rattan, "Rainfall Prediction using Random Forest and XGBoost-A comparative study," Proceedings of the 2024 5th International Conference on Data Intelligence and Cognitive Informatics (ICDICI), pp. 1–6, 2024.

[2] J. Puttanapura and C. K. K. Reddy, "Improvised Prediction of Rainfall using Random Forest Classifier Over Taiwan," Proceedings of the 2024 4th International Conference on Soft Computing for Security Applications (ICSCSA), pp. 1–6, 2024.

[3] A. Jain, "Enhancing Weather Prediction Accuracy with Gradient Boosting: A Machine Learning Approach," Proceedings of the 2025 2nd International Conference on Computational Intelligence, Communication Technology and Networking (CICTN), pp. 1–5, 2025.

[4] P. Sahu, S. Kumar, and R. K. Yadav, "Explainable AI-Powered and Machine Learning Approach for Heavy Rainfall Prediction," Proceedings of the 2025 12th International Conference on Reliability, Infocom Technologies and Optimization (ICRITO), pp. 1–6, 2025.

[5] R. P. Utture, A. K. Biju, Y. V. S. Murthy, L. R. Cenkeramaddi, and N. Kumari, "Weather Prediction based on the Cloud Images using Machine and Deep Learning Algorithms," Proceedings of the 2025 International Conference on Next Generation Communication & Information Processing (INCIP), pp. 1–5, 2025.

- [6] W. Zou, J. Ji, Y. Wang, J. Wang, Y. Qian, and J. Liu, "Convolutional LSTM with Self-Attention Mechanism for Extreme Weather Prediction," *Proceedings of the International Conference on Artificial Intelligence and Information Systems*, pp. 1–8, 2024.
- [7] Y. Liu and S. Cai, "Advancements in Weather Forecasting: A Review of Satellite Imagery and AI Integration," *Proceedings of the International Conference on Intelligent Informatics and BioMedical Sciences (ICIIBMS)*, pp. 1–7, Nov. 2024.
- [8] A. Saxena, N. Verma, and K. C. Tripathi, "A Review Study of Weather Forecasting Using Artificial Neural Network Approach," *International Journal of Engineering Research & Technology (IJERT)*, vol. 2, no. 11, pp. 2029–2032, Nov. 2013.
- [9] M. M. Hassan *et al.*, "Machine Learning-Based Rainfall Prediction: Unveiling Insights and Forecasting for Improved Preparedness," *IEEE Access*, vol. 11, pp. 132196–132222, 2023.
- [10] A. E. Abdishakur, A. M. Tahlil, and A. A. Aden, "Hybrid Machine Learning Model for Rainfall Prediction Using Time-Series Data," *International Journal of Engineering Trends and Technology (IJETT)*, vol. 73, no. 6, pp. 17–29, June 2025.
- [11] B. K. Cahyono *et al.*, "Leveraging machine learning and open accessed remote sensing data for precise rainfall forecasting," *Communications in Science and Technology*, vol. 10, no. 1, pp. 135–147, 2025.
- [12] A. Rahman *et al.*, "Rainfall Prediction System Using Machine Learning Fusion for Smart Cities," *Sensors*, vol. 22, no. 9, art. no. 3504, pp. 1–15, May 2022.
- [13] L. Bhaskar, S. M. N, V. S, and T. Anand, "Rainfall Prediction Using Machine Learning," *International Journal of Advanced Research (IJAR)*, vol. 13, no. 5, pp. 1211–1216, May 2025.
- [14] M. M. Hassan *et al.*, "Machine Learning-Based Rainfall Prediction: Unveiling Insights and Forecasting for Improved Preparedness," *IEEE Access*, vol. 11, pp. 132196–132222, 2023.
- [15] A. E. Abdishakur, A. M. Tahlil, and A. A. Aden, "Hybrid Machine Learning Model for Rainfall Prediction Using Time-Series Data," *International Journal of Engineering Trends and Technology (IJETT)*, vol. 73, no. 6, pp. 17–29, June 2025.
- [16] B. K. Cahyono *et al.*, "Leveraging machine learning and open accessed remote sensing data for precise rainfall forecasting," *Communications in Science and Technology*, vol. 10, no. 1, pp. 135–147, 2025.
- [17] A. Rahman *et al.*, "Rainfall Prediction System Using Machine Learning Fusion for Smart Cities," *Sensors*, vol. 22, no. 9, art. no. 3504, pp. 1–15, May 2022.
- [18] L. Bhaskar, S. M. N, V. S, and T. Anand, "Rainfall Prediction Using Machine Learning," *International Journal of Advanced Research (IJAR)*, vol. 13, no. 5, pp. 1211–1216, May 2025.

Appendix

Firmware Code

```
#include <Wire.h>
#include "Adafruit_SHT31.h"
#include <ESP8266WiFi.h>
#include <WiFiClientSecure.h>

// Initialize the SHT31/SHT30 sensor
// Default I2C address is 0x44
// --- Configuration ---
const char* ssid = "Ranjit-AP";
const char* password = "AgS@2004!#$";

// The ID from your Google Web App URL (the part between /s/ and /exec)
const char* GScriptId = "AKfycby6Q1Z48GAdaxzs4qmC0nepOfRn-83mwZaurGeRjy9wDHqHECcdMlI9gQk2BgqeapU1";

const char* host = "script.google.com";
const int httpsPort = 443;

Adafruit_SHT31 sht31 = Adafruit_SHT31();
/*void setup() {
  Serial.begin(9600); // Use 115200 to avoid gibberish
  sht31.begin();

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi Connected!");
}*/

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi Connected!");
  while (!Serial) delay(10); // Wait for serial console to open

  Serial.println("SHT30 Test Started!");

  // Start I2C on NodeMCU (D2=SDA, D1=SCL)
  Wire.begin(4, 5);

  if (!sht31.begin(0x44)) { // Check for sensor at address 0x44
    Serial.println("Couldn't find SHT30! Check wiring.");
  }
```

```

        while (1) delay(1);
    }
}

void loop() {
    float t = sht31.readTemperature();
    float h = sht31.readHumidity();

    if (!isnan(t) && !isnan(h)) { // check if 'is not a number'
        Serial.print("Temp *C = "); Serial.print(t);
        Serial.print("\t\tHum % = "); Serial.println(h);
    } else {
        Serial.println("Failed to read temperature or humidity");
    }
    sendData(t, h);
    //delay(1800000UL); // Wait 30 min between readings
    delay(6000);
}

void sendData(float temp, float hum) {
    WiFiClientSecure client;
    client.setInsecure(); // Required for Google Scripts

    if (!client.connect(host, httpsPort)) {
        Serial.println("Connection failed");
        return;
    }

    String url = String("/macros/s/") + GScriptId + "/exec?temp=" +
String(temp) + "&hum=" + String(hum);

    client.print(String("GET ") + url + " HTTP/1.1\r\n" +
        "Host: " + host + "\r\n" +
        "User-Agent: BuildFailureDetectorESP8266\r\n" +
        "Connection: close\r\n\r\n");

    Serial.println("Data sent to Google Sheets!");
}

```

Code Explanation

The hardware system is developed using a NodeMCU ESP8266 microcontroller and an SHT30 temperature and humidity sensor. The program reads environmental data at regular intervals and uploads it to cloud storage for real-time monitoring.

Step 1: Library Inclusion and Configuration

- Import the required libraries for I2C communication, sensor interfacing, Wi-Fi connectivity, and secure internet communication.
- Configure the Wi-Fi network details and the web service address used to upload the sensor readings.
- These settings allow the microcontroller to connect to the internet and communicate with the cloud platform.

Step 2: System Initialization

- Establish a connection with the available Wi-Fi network.
- Initialize the I2C communication required for sensor operation.
- Detect the connected temperature and humidity sensor before starting data collection.
- If the sensor is not detected, the program stops to prevent incorrect measurements.

Step 3: Sensor Data Collection

- Read the temperature and humidity values from the sensor at regular time intervals.
- Check whether the collected values are valid before processing them.
- Display the measured values on the serial monitor for monitoring and debugging purposes.

Step 4: Data Upload

- Create a secure internet connection with the cloud server.
- Send the temperature and humidity readings to the online database using an HTTP request.
- Store the received data so that it can be accessed later by the web application for real-time visualization and analysis.

Github Link

<https://github.com/AGNIK321/Weather-Forecasting-GDB>

Plagiarism Analysis Report

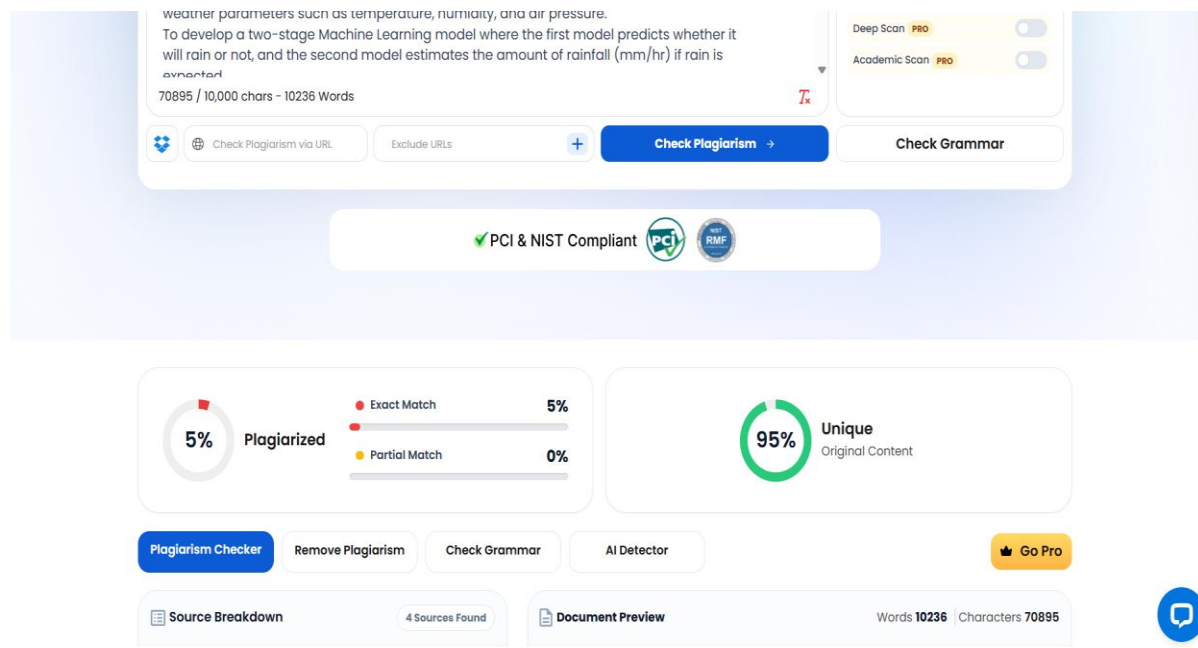


Fig. 17: Plagiarism Analysis Report Showing 95% Original Content and 5% Similarity